Contents

The C Piscine Final Exam Guide 2
The promise and the rule 2
How to use this guide 2
Part I — The toolkit 4
Man pages and system headers 4
Compiling with both compilers 5
Reading compiler errors and warnings 7
Debuggers: gdb and lldb as peers 11
Valgrind and the sanitizers 16
Testing yourself 18
Vim survival minimum 22
Practicing from a blank file 23
Part II — Foundations 25
Output and write 25
Control flow 29
Characters and ASCII 32
Pointers and arrays 34
Strings 37
Part III — Numbers and recursion 42
Numbers, parsing and overflow in int 42
Bases and bits 46
Recursion 48
Part IV — Programs, memory and builds 54
Programs and argv 54
The heap and ownership 56
Headers, Makefiles and libraries 60
File descriptors and read 64
Part V — Data structures and algorithms 69
Function pointers 69
Sorting and searching 73
Structs, lists and trees 77
Parsers and state machines 83
Part VI — Appendices 88
Appendix A — Pre-submit checklist 88
Appendix B — Universal edge-case matrix 88
Appendix C — GDB and LLDB card 89
Appendix D — Compiler and sanitizer card 89
Appendix E — Valgrind decoder 90

Appendix F — Reference-tool card 91
Appendix G — Glossary 91
Appendix H — Sources and trust methodology 92

42 · C PISCINE

The C Piscine Final Exam Guide

For: pisciners at any point of the C Piscine or of Piscine Reloaded. The exams ask what the projects ask, so this guide teaches what the projects ask — the concepts — and the local tools you write and debug C with.

The promise and the rule

This guide contains zero solutions to any exercise of the Piscine, Piscine Reloaded or the Common Core. That is the design, not caution. Every complete program in these pages solves an invented task that sits next to a real one: same skills, different task. Known algorithms — Euclid's gcd, a simple sort, binary search — are explained as knowledge: how they work, why, and what they cost, never fitted to a named exercise.

Here is why that helps you. Reading an idea built on a neighbor task does not use up an exercise you still need to practice. Then you open a blank file and build your own with nothing to lean on, and that retrieval — rebuilding the idea yourself — is what makes it stay.

Your subject is the law: its file names, prototypes, allowed-functions line and examples decide what your code must be. This guide never overrides them, and when it quotes a subject it names which one. Its own examples call libc freely — `printf`, `strlen`, `atoi` in a test driver — because they are never turned in; yours call only what your subject allows.

If you catch yourself hunting for a finished answer to memorize, you are training the wrong muscle, and this guide keeps declining to hand you one. It hands you concepts, tools, traps, and ways to check your work byte for byte.

How to use this guide

Start with Part I, but not all of it at once. In your first weeks, read Man pages and system headers, Compiling with both compilers, Reading compiler errors and warnings, Hunts 1 and 2 of Debuggers: gdb and lldb as peers, Testing yourself (leave Reference tools and the valgrind loop at its end for later), Vim survival minimum and Practicing from a blank file. Then go to Part II. Hunts 3 and 4 and Valgrind and the sanitizers are about memory from `malloc`: come back to them after The heap and ownership. Parts II to V follow the themes in the order the Piscine's modules build them, from output and loops (C 00) through pointers, strings, numbers, recursion, argv and the heap to lists and trees (C 12–C 13), with parsing last. Part VI holds the cards to keep open beside you.

Every concept chapter has six parts: Concepts; a Worked example, an invented task with full code and a real run; Questions to ask, which fit any task of the theme; a Self-check, where you answer before you open the answer (the PDF and the page beside this file print the answers at the end of

the chapter); Pitfalls; and Practice and check, a task given as a spec only, with the tool recipes that check your version.

Reading is half the work: Practicing from a blank file gives the loop that makes a chapter stick. A word you do not know is probably in Appendix G — Glossary.

Before any of it, the subject, `man` and the peer next to you come first. That is how the Piscine is meant to work, and explaining your code to a peer is practice too.

Reading the examples

Most examples are whole programs that read their input from the command line. Until Programs and argv explains it, three facts are enough. `argc` counts the words on the command line, the program's name included. `argv[1]` is the first word after the name and `argv[2]` the second; each is a string you index like any other, so `argv[1][0]` is its first byte. When no word was typed, `argv[1]` is NULL, so test `argc` before using it. Test drivers also call libc freely (`printf` , `atoi`); your turn-in may not.

Every output here was produced on one machine: Ubuntu 22.04, gcc 10.5.0, clang 12.0.1 (what `cc` runs there), GNU gdb 12.1, valgrind 3.18.1, GNU Make 4.3, norminette 3.3.58, and dash as `/bin/sh` . The lldb lines were checked with lldb 12.0.1 from Ubuntu's `lldb-12` package, which installs the command as `lldb-12` . Messages change between versions; check yours before you compare word for word:

```
cc --version
gcc --version
gdb --version
lldb --version || lldb-12 --version
valgrind --version
```

Part I — The toolkit

Everything in this part runs on your own machine, and every later chapter uses it.

Man pages and system headers

Your machine documents itself: the man pages are installed with the tools they describe, and every prototype your code needs sits in a header under `/usr/include`.

Sections

`man` has numbered sections, and the number matters: 1 is shell commands, 2 system calls, 3 C library functions, 7 overviews. `man write` lands on `WRITE(1)`, "send a message to another user". `man -f` lists every page of a name:

```
$ man -f write
write (1)            - send a message to another user
write (1posix)       - write to another user
write (3posix)       - write on a file
write (2)            - write to a file descriptor
```

The `posix` pages describe the POSIX standard's version; the plain numbers, your system's. `man printf` sets the same trap: it opens the shell command.

- `man 2 write`: the system call, with its header, prototype and return value.
- `man 3 malloc`: malloc, free, calloc and realloc on one page.
- `man 7 ascii`: every character's code in octal, decimal and hex.
- `man isspace`: opens isalpha(3), every is* test on one page.
- `man -k alpha`: forgot a name? It searches all names and one-line summaries.

Reading a page

Section 2 and 3 pages share one skeleton: NAME; SYNOPSIS, with the `#include` line and the exact prototype; DESCRIPTION; RETURN VALUE; and, on pages for calls that can fail with an `errno` code, ERRORS (`man 2 write` has one, `man 3 strlen` does not). The RETURN VALUE of `man 2 write` holds two facts that matter more than they look: "On error, -1 is returned", and "a successful write() may transfer fewer than count bytes".

In the pager, `space` goes down a screen, `/word` and Enter search, `q` quits. This machine's pager (`more`) cannot find the bold headings — `/ERRORS` answers `Pattern not found` — so let `grep` find a section:

```
man 2 write | grep -n -A3 'RETURN VALUE'
```

The headers themselves

Headers settle what pages leave vague:

```
grep -n -B1 "define INT_MAX" /usr/include/limits.h
echo '#include <limits.h>' | cc -E -dM - | grep -w -E 'INT_(MAX|MIN)'
```

The first prints line 80, defining `INT_MIN` as `(-INT_MAX - 1)`, above line 81, defining `INT_MAX` as `2147483647`: the positive twin of the smallest `int` does not fit in an `int`. The second asks the compiler, which can bring its own copy of a header: `-E` stops after the preprocessor, `-dM` prints the macros instead of the code, `-` reads standard input. clang answers `#define INT_MIN (-__INT_MAX__ -1)`.

Prototypes are found the same way. `grep -n 'ssize_t write (' /usr/include/unistd.h` prints line 378, the prototype the man page shows, `extern ssize_t write (int __fd, const void *__buf, size_t __n) __wur`, with attributes for the compiler on the next line.

Self-check

Q1. You want the prototype of the system call `read`. Why is `man read` the wrong command?

Answer: end of the chapter

Q2. Where does a man page say what a call gives back when it fails, and where do you learn why it failed?

Answer: end of the chapter

Self-check answers

> **Answer to Q1**
>
> With no number, `man` opens the first section that has the name: here `read(1posix)`, the shell's `read`. `man 2 read` is the system call.

> **Answer to Q2**
>
> RETURN VALUE (for `write`, -1). The reason is left in `errno`, and ERRORS lists the codes it can hold — `ENOSPC` when the device has no room for the data.

Compiling with both compilers

Your subject sets the flags. BSQ's subject, for one, says your program "must compile using cc with the following flags: -Wall -Wextra -Werror". `-Wall` and `-Wextra` turn on most warnings; `-Werror` makes each one an error, so one warning means no program. Add `-g` while you work, so the debuggers can show file names and line numbers:

```
cc -Wall -Wextra -Werror -g strip_letter.c -o strip_letter
```

`strip_letter.c` is the specimen of Debuggers: gdb and lldb as peers. An alias saves typing, but it lives only in the shell you typed it into; a new `sh` answers `sh: 1: mycc: not found`.

```
alias mycc='cc -g -Wall -Wextra -Werror'
mycc strip_letter.c -o strip_letter
```

To keep it, put the `alias` line in your interactive shell's startup file: for bash, `~/.bashrc`, which every new interactive bash reads (a plain `sh` does not).

Two compilers

`cc` is a name, not a compiler: `cc --version` tells you which one answers. On the machine behind these pages it is clang 12.0.1, and `gcc` is gcc 10.5.0. Their warnings differ, so build with both:

```
gcc -Wall -Wextra -Werror -g strip_letter.c -o strip_letter
clang -Wall -Wextra -Werror -g strip_letter.c -o strip_letter
```

Reading compiler errors and warnings shows the differences: clang alone flags a variable set only inside an `if`; gcc alone, and only when it optimizes, flags one set only inside a loop; gcc alone refuses `-lm` written before the file that needs it.

Files without a main

C 00's subject says you submit a `main()` only when it asks for a program. A file of functions alone does not link, since nothing in it is `main`; `-c` stops after compiling and writes an object file. To run the functions, put a test `main` in a file of its own. Save the function as `twice.c`:

```
int twice(int n)
{
    return (n * 2);
}
```

and the test driver as `main.c`:

```
#include <stdio.h>

int twice(int n);

int main(void)
{
    printf("%d\n", twice(21));
    return (0);
}


cc -Wall -Wextra -Werror -c twice.c
cc -Wall -Wextra -Werror twice.c main.c -o twice_test
./twice_test
```

The first line writes `twice.o`; the last prints `42`.

The sanitizer build

For testing, add the sanitizers:

```
cc -Wall -Wextra -Werror -g -fsanitize=address,undefined strip_letter.c -o san
```

At run time they catch out-of-bounds writes and undefined behavior (an operation the C standard gives no meaning to, such as an `int` sum too large for `int`; after it the program may print anything, crash or look fine), and name the line. Not everything: a read of uninitialized heap memory passes them in silence, and a leak is caught only some of the time. And on this machine `cc` is clang, whose reports show bare addresses where gcc's name the line. Valgrind and the sanitizers covers all three, and why a sanitizer build never runs under valgrind.

The style checker

`norminette` checks a file against the Norm, 42's coding style. Three of its messages meet everyone early. `INVALID_HEADER`: the 42 header comment is missing (this guide never prints it, so every example here gets this one). `FORBIDDEN_CS`: a control structure the Norm forbids (Control flow lists them). `TOO_MANY_LINES`: a function body over 25 lines — split out a helper.

```
norminette twice.c main.c
```

Reading compiler errors and warnings

A screen of red is a list, read from the top. Every diagnostic has the same anatomy:

```
diag.c:9:2: error: implicit declaration of function 'write' is invalid in C99 [-Werror,-Wimplicit-
function-declaration]
```

File, line, column, severity, message, and in brackets the flag behind it. Under `-Werror` every warning says `error:`, and the bracket shows it began as a warning. A line starting with `note:` belongs to the message above it.

A file of mistakes

This file is deliberately broken — a specimen, not a task. Each function holds one mistake (`report` holds two), and `sign_of` is meant to give -1, 0 or 1. Save it as `diag.c`:

```
#include <stdio.h>
#include <stdlib.h>

void    say_digit(int d)
{
    char    c;

    c = '0' + d;
    write(1, c, 1);
}

int count_blanks(char *s, int limit)
{
    unsigned int    i;
    int             n;
    int             spare;

    i = 0;
    n = 0;
    while (s[i] && i < limit)
    {
        if (s[i] = ' ')
            n++;
        i++;
    }
    return (n);
}

int sign_of(int x)
{
    int r;

    if (x < 0)
        r = -1;
    return (r);
}

int under_ten(int x)
{
    if (x < 10)
        return (1);
}

void    report(int n, int width)
{
    printf("%s\n", n);
}
```

Keep the first line of each message (`2>&1` sends them into the pipe):

```
$ cc -Wall -Wextra -Werror -c diag.c 2>&1 | grep 'error:'
diag.c:9:2: error: implicit declaration of function 'write' is invalid in C99 [-Werror,-Wimplicit-
function-declaration]
diag.c:22:12: error: using the result of an assignment as a condition without parentheses [-Werror,-
Wparentheses]
diag.c:16:9: error: unused variable 'spare' [-Werror,-Wunused-variable]
diag.c:20:19: error: comparison of integers of different signs: 'unsigned int' and 'int' [-Werror,-
Wsign-compare]
diag.c:33:6: error: variable 'r' is used uninitialized whenever 'if' condition is false [-Werror,-
Wsometimes-uninitialized]
diag.c:42:1: error: non-void function does not return a value in all control paths [-Werror,-
Wreturn-type]
diag.c:46:17: error: format specifies type 'char *' but the argument has type 'int' [-Werror,-
Wformat]
diag.c:44:24: error: unused parameter 'width' [-Werror,-Wunused-parameter]
```

gcc finds seven of the eight, in its own words, and says nothing about line 33. Neither list is in line order. (`norminette` objects to line 22 as well: `ASSIGN_IN_CONTROL` .)

First message first

Fix the first message, then compile again. Line 9 calls `write` , never declared: line 2 includes the wrong header, and the SYNOPSIS of `man 2 write` names the right one. Fix line 2, and line 9 gets a new message:

```
$ cc -Wall -Wextra -Werror -c diag.c 2>&1 | grep 'diag.c:9'
diag.c:9:11: error: incompatible integer to pointer conversion passing 'char' to parameter of type
'const void *' [-Werror,-Wint-conversion]
```

Without a prototype the compiler could not check the arguments; with one, it sees a `char` where `write` wants an address. One message can hide another, so fix the first, recompile, and read the new list.

Silencing is not fixing

clang's message for line 33 ends with three notes. The last:

```
diag.c:31:7: note: initialize the variable 'r' to silence this warning
        int     r;
                ^
                 = 0
```

Do that and the message is gone, but `sign_of(5)` returns 0: the positive case is still missing. A note says how to quiet the compiler, not what your function owes. `(void)width;` quiets line 44 the same way: right when your subject's prototype hands you a parameter you do not need, wrong when the warning pointed at a bug (the specimen of Debuggers: gdb and lldb as peers has one).

gcc runs some checks only when it optimizes (`-O2` asks for optimization). A variable set only inside a loop, then returned, passed clang at `-O0` and `-O2` and plain gcc; `gcc -O2` answered `may be used uninitialized in this function` . Add an `-O2` build to your checks.

Linker errors

When every file compiles, the linker joins them and can still refuse. Its messages come from `/usr/bin/ld`, not from the compiler. They have no column, no `error:` with a flag in brackets, and the build ends with `ld returned 1 exit status` (gcc) or `linker command failed` (clang). Without `-g` they name an offset such as `main.c:(.text+0x0)`; with `-g`, the file and line of the call. Leave a test `main` inside `twice.c` and build it with `main.c`:

```
$ cc -Wall -Wextra -Werror twice.c main.c -o twice_test
/usr/bin/ld: /tmp/main-b56066.o: in function `main':
main.c:(.text+0x0): multiple definition of `main'; /tmp/twice-2945fb.o:twice.c:(.text+0x10): first
defined here
clang: error: linker command failed with exit code 1 (use -v to see invocation)
```

A program has one `main`: keep your test `main` in its own file, and hand in only the files your subject names.

The other classic, `undefined reference`, means something was called but nothing on the line defines it: a file left out, or a library in the wrong place. `wave.c` calls `cos` from the math library, `-lm`, on `argc` so the compiler cannot compute it in advance:

```
#include <math.h>
#include <stdio.h>

int main(int argc, char **argv)
{
    (void)argv;
    printf("%f\n", cos(argc));
    return (0);
}
```

```
$ gcc -Wall -Wextra -Werror -lm wave.c -o wave
/usr/bin/ld: /tmp/cciWW4hx.o: in function `main':
wave.c:(.text+0x27): undefined reference to `cos'
collect2: error: ld returned 1 exit status
$ gcc -Wall -Wextra -Werror wave.c -lm -o wave
$ ./wave
0.540302
```

gcc here tells the linker to keep a library only if something before it on the line needs it, so `-lm` goes after the files; clang 12 links both spellings.

Self-check

Q1. A message ends in `[-Werror,-Wunused-parameter]`. Compile `diag.c` once with `-Wall -Werror -c` and once with `-Wall -Wextra -Werror -c`. Which of the flags you typed turned that check on, and which one stopped the build?

Answer: end of the chapter

Q2. clang suggests `= 0` for `r` in `sign_of`. Before taking the advice: which values of `x` reach `return (r);` without passing through `r = -1;`, and what should each of them return?

Answer: end of the chapter

Q3. `undefined reference to 'twice'` appears. Compiler or linker, and where do you look first?

Answer: end of the chapter

Self-check answers

> **Answer to Q1**
>
> `-Wextra`: with `-Wall` alone, neither compiler reports line 44 of `diag.c`. `-Werror` made the warning an error.

> **Answer to Q2**
>
> Every `x` from 0 up. 0 should give 0 and every positive value 1, so no single starting value is right for both. The note silences the warning, and the missing case is still missing.

> **Answer to Q3**
>
> The linker: the message comes from `/usr/bin/ld`, and the build ends with the linker's exit status. Look at the build line first: is the file that defines `twice` on it?

Debuggers: gdb and lldb as peers

A debugger turns "it segfaulted" into "line 12, `src` is NULL". gdb and lldb are peers here: every move below exists in both, and Appendix C pairs their commands. The sessions ran under GNU gdb 12.1; the lldb commands were checked with lldb 12.0.1 (installed as `lldb-12`).

The specimen

You learn a debugger by hunting, so here is prey, on an invented task: copy `argv[1]` into a new heap buffer, minus every occurrence of the first character of `argv[2]`, so `./strip_letter banana a` should print `bnn`. Four bugs hide in it. Save it as `strip_letter.c`; line numbers count the first `#include` as line 1.

Two things in it may be new. `argv[1]` and `argv[2]` are the first and second words typed after the program's name; Programs and argv covers them, and Reading the examples in How to use this guide gives the three facts you need now. `malloc(len)` asks for `len` bytes of memory while the program runs and returns their address, or NULL when it cannot. Those bytes stay yours until `free` gives them back; that memory is the heap, which The heap and ownership covers. Hunts 1 and 2 need nothing more.

```
#include <unistd.h>
#include <stdlib.h>

char    *strip_letter(char *src, char cut)
{
    char    *out;
    int     len;
    int     i;
    int     j;

    len = 0;
    while (src[len])
        len++;
    out = malloc(len);
    if (!out)
        return (NULL);
    i = 0;
    j = 0;
    while (i < len - 1)
    {
        if (src[i] != cut)
        {
            out[j] = src[i];
            j++;
        }
        i++;
    }
    out[j] = '\0';
    return (out);
}

int main(int argc, char **argv)
{
    char    *clean;
    int     i;

    (void)argc;
    clean = strip_letter(argv[1], argv[2][0]);
    if (!clean)
        return (1);
    i = 0;
    while (clean[i])
        i++;
    write(1, clean, i);
    write(1, "\n", 1);
    return (0);
}
```

`strip_letter` measures `src` (lines 11–13), allocates a buffer (line 14), copies every character that is not `cut` (lines 17–27) and terminates the copy (line 28). `main` casts `argc` to void (line 37 — remember that cast), calls the worker (line 38) and prints the result. It compiles clean with gcc and clang under `-Wall -Wextra -Werror -g`, and it runs:

```
$ ./strip_letter banana a | cat -e
bnn$
$ ./strip_letter world o | cat -e
wrl$
```

The first answer is right; the second is not (`wrld` was owed), and two more bugs have not surfaced. The compiler checks form, not meaning.

Hunt 1: the segfault and the backtrace

Run it with no arguments:

```
$ ./strip_letter
Segmentation fault
```

Ask the debugger where it died. `-q` skips the banner; the start-up lines after `run` are trimmed:

```
$ gdb -q ./strip_letter
Reading symbols from ./strip_letter...
(gdb) run

Program received signal SIGSEGV, Segmentation fault.
0x0000000000401161 in strip_letter (src=0x0, cut=83 'S') at strip_letter.c:12
12      while (src[len])
(gdb) bt
#0  0x0000000000401161 in strip_letter (src=0x0, cut=83 'S')
    at strip_letter.c:12
#1  0x000000000040124e in main (argc=1, argv=0x7ffe0a7b8da8)
    at strip_letter.c:38
(gdb) print src
$1 = 0x0
```

Read the report like a sentence: it died at line 12, inside `strip_letter`, with `src=0x0` — NULL — printed right there. `bt` (backtrace) shows the call chain: `main` passed that NULL from line 38, where `argv[1]` is the NULL that ends the list because `argc=1`; `cut=83 'S'` is a byte read from past the end of the argument list. Many hunts end here, in the argument values of the first frame. The fix is a guard that uses `argc` instead of silencing it: replace `(void)argc;` on line 37 with `if (argc != 3)` and, on a new line under it, `return (1);`. What a guard prints is your subject's call; this invented task names no output, so this one prints nothing.

The same hunt in lldb:

```
$ lldb ./strip_letter
(lldb) run
(lldb) bt
(lldb) frame variable src
```

`run` stops at line 12 on signal SIGSEGV, fault address 0x0; `bt` shows frame #0 `strip_letter`, #1 `main`, then libc's frames; `frame variable src` prints `(char *) src = 0x0000000000000000`. Type each command bare: lldb reads no `#` note after one, and hands its words to the command, so a note after `run` reaches the program as arguments.

Where lldb comes from a versioned package, as in this repository's container, the command carries the number: `lldb-12 ./strip_letter`. `which lldb lldb-12` shows which one you have; if it prints nothing, lldb is not installed, and gdb does everything this chapter shows. lldb 12 may first print `unsupported DW_FORM value: 0x1f` warnings about the system's libraries; they are harmless, and your own file's lines and variables still show.

If `run` answers `error: 'A' packet returned an error: 8`, your container forbids switching off address randomization: type `settings set target.disable-aslr false`, then `run` again.

Hunt 2: the wrong answer and the breakpoint

With the guard in, `world o` still prints `wrl; banana a` printing the right answer proved nothing. No crash, so plant a breakpoint where the copy finishes (line 28) and look at the state (start-up lines trimmed):

```
$ gdb -q ./strip_letter
Reading symbols from ./strip_letter...
(gdb) break strip_letter.c:28
Breakpoint 1 at 0x401202: file strip_letter.c, line 28.
(gdb) run world o

Breakpoint 1, strip_letter (src=0x7fffea1b4157 "world", cut=111 'o') at strip_letter.c:28
28      out[j] = '\0';
(gdb) print i
$1 = 4
(gdb) print len
$2 = 5
(gdb) print src[i]
$3 = 100 'd'
```

The loop stopped with `i` at 4 while `len` is 5, and `src[i]` is the `d` never copied: line 19, `while (i < len - 1)`, quits one character early. The fix is `while (i < len)`.

Before fixing it, confirm it with a breakpoint that carries a condition. In a fresh session, ask whether the loop ever looks at the `d`:

```
(gdb) break strip_letter.c:21 if src[i] == 'd'
Breakpoint 1 at 0x4011bd: file strip_letter.c, line 21.
(gdb) run world o
wrl
[Inferior 1 (process 9042) exited normally]
```

It never stops: line 21 never saw the `d`. A function takes a condition too (`break strip_letter if cut == 'o'`), and Recursion uses that form.

When you cannot see who changes a variable, watch it: the program halts on every write to it. Break at line 19, `run world o`, then (trimmed to the watch):

```
(gdb) watch j
Hardware watchpoint 2: j
(gdb) continue
Continuing.

Hardware watchpoint 2: j

Old value = 0
New value = 1
strip_letter (src=0x7ffcfc859157 "world", cut=111 'o') at strip_letter.c:26
26          i++;
```

It stopped right after line 24 wrote `j`. In lldb: `b strip_letter.c:28`, `run world o`, `p i`, `p len`, `p src[i]`; the condition is `br s -f strip_letter.c -l 21 -c "src[i] == 'd'"`, and the watchpoint `watchpoint set variable j`.

Hunt 3: the invisible overflow

Needs The heap and ownership.

Now a word with no letter to remove:

```
$ ./strip_letter world z | cat -e
world$
```

Looks right; it is not. When nothing is removed, `j` reaches `len`, and line 28 writes the terminator at `out[len]` — one byte past the `malloc(len)` block of line 14. Nothing crashes, yet the byte lands on memory the program never asked for. This is sanitizer work. Rebuild with gcc, whose reports name the lines here, and run the same input (trimmed to the lines that matter, each path cut to its file name):

```
$ gcc -Wall -Wextra -Werror -g -fsanitize=address,undefined strip_letter.c -o san
$ ./san world z
=================================================================
==9088==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x502000000015 at pc 0x55db7ce51663
bp 0x7fff6a3fa9b0 sp 0x7fff6a3fa9a0
WRITE of size 1 at 0x502000000015 thread T0
    #0 0x55db7ce51662 in strip_letter strip_letter.c:28
    #1 0x55db7ce517cf in main strip_letter.c:39

0x502000000015 is located 0 bytes to the right of 5-byte region [0x502000000010,0x502000000015)
allocated by thread T0 here:
    #0 0x7f9cd0ba8887 in __interceptor_malloc ../../../../src/libsanitizer/asan/
asan_malloc_linux.cpp:145
    #1 0x55db7ce51395 in strip_letter strip_letter.c:14
```

Read it as three answers: what happened (a 1-byte write right past a 5-byte block on the heap), where (line 28), and where that block was born (line 14). `main` sits at line 39 now: Hunt 1's guard took two lines where the cast took one. The fix is the terminator's byte: `malloc(len + 1)`.

Hunt 4: the leak

Needs The heap and ownership.

Three fixes in, the output is right and a bug remains: `clean` is never freed. No symptom, no crash; valgrind's exit report tells you (trimmed; Valgrind and the sanitizers covers the tool):

```
$ valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./strip_letter world z
==9101== 6 bytes in 1 blocks are definitely lost in loss record 1 of 1
==9101==    at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==9101==    by 0x401186: strip_letter (strip_letter.c:14)
==9101==    by 0x401263: main (strip_letter.c:39)
```

Do not count on the sanitizer build for this one: Valgrind and the sanitizers shows gcc's and clang's builds disagreeing about it. The fix is `free(clean);` before `main` returns; valgrind then ends with `All heap blocks were freed -- no leaks are possible.`

Four bugs, four detectors: a backtrace, a breakpoint, a sanitizer, a leak report. That mapping — symptom to tool — is the skill.

The command card

Running with arguments, breakpoints with and without a condition, stepping, printing (plain and in hex), raw memory, locals, frames, watchpoints: Appendix C pairs each gdb command with its lldb twin.

Both have a full-screen mode. `gdb -tui ./prog` (or `Ctrl-x a` inside a session) opens a source panel that follows execution; if the panel gets garbled, `Ctrl-l` redraws it. lldb's is the `gui` command.

The segfault triage drill

When a crash hits, these five steps usually name its line and its cause:

1. Reproduce it with the exact argument list that crashed.
2. Rebuild with `-g`, keeping `-Wall -Wextra -Werror` on.
3. `gdb ./prog`, then `run` with those arguments.
4. `bt`, and read the innermost frame that is in your file: the line, then the argument values — a `0x0` there is often the whole story.
5. `print` the variables on that line. If the cause is still hidden, `break` one line earlier, run again, and step with `next` while printing.

Valgrind and the sanitizers

Needs The heap and ownership.

Two kinds of run-time checkers watch memory. Sanitizers are compiled into your program: `-fsanitize=address` (ASan) and `-fsanitize=undefined` (UBSan), built as Compiling with both compilers shows. Valgrind runs your unchanged program on a synthetic CPU (`man valgrind`) and checks every byte it touches. Each sees bugs the others miss, so run both, but on separate builds. Under valgrind, gcc's sanitizer build stopped before `main` (`ASan runtime does not come first in initial library list`), and valgrind still closed with `ERROR SUMMARY: 0 errors`.

Valgrind on the specimen

One invocation serves every run in this guide; Appendix E is its card, and an alias (Compiling with both compilers) saves retyping it:

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./prog args
```

Here it runs on the specimen from Debuggers: gdb and lldb as peers, with the fixes of Hunts 1 and 2 in and the `malloc` still one byte short (trimmed):

```
$ valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./strip_letter world z
==4447== Invalid write of size 1
==4447==    at 0x10921F: strip_letter (strip_letter.c:28)
==4447==    by 0x109270: main (strip_letter.c:39)
==4447==  Address 0x4a8d045 is 0 bytes after a block of size 5 alloc'd
==4447==    at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==4447==    by 0x1091AB: strip_letter (strip_letter.c:14)
==4447==
==4447== Invalid read of size 1
==4447==    at 0x10929D: main (strip_letter.c:43)
```

The same three answers as the sanitizer's in Hunt 3: what (a 1-byte write right past a 5-byte block), where (line 28, called from 39), and where the block was born (line 14). The `Invalid read` is the same stray byte read back by `main`: one bad write, two reports, so fix the first one first. With the `malloc` fixed, only Hunt 4's leak remains, and its report names line 14: where the block was born, not where it was lost. With `free(clean);` added, the run ends with `ERROR SUMMARY: 0 errors`.

`--show-leak-kinds=all` prints a record for every kind of leak: `definitely lost` (nothing points at the block), `indirectly lost` (only a lost block points at it), `still reachable` (a pointer, such as a global, still holds it at exit). Without the flag, the last two appear only as counts in `LEAK SUMMARY`. A toy that lost a struct holding a `malloc`'d field showed `8 bytes in 1 blocks are indirectly lost` beside `24 (16 direct, 8 indirect) bytes in 1 blocks are definitely lost`.

For scripts, `-q` prints only errors and `--error-exitcode=N` turns any error into status N, but a leak counts only under `--leak-check=full`. On the leaky build:

```
$ valgrind -q --error-exitcode=99 ./strip_letter world z; echo $?
world
0
$ valgrind -q --leak-check=full --error-exitcode=99 ./strip_letter world z > /dev/null 2>&1; echo $?
99
```

Who sees what

Each cell comes from a toy run here with gcc and with clang, which agreed on every row: ASan and UBSan each built alone, valgrind on a plain build.

Bug	ASan	UBSan	valgrind
Write past a `malloc` block	`heap-buffer-overflow`	silent	`Invalid write`
Write past a local array	`stack-buffer-overflow`	`index 4 out of bounds`	silent
Read after `free`	`heap-use-after-free`	silent	`Invalid read`
`free` twice	`attempting double-free`	silent; libc aborts	`Invalid free()`
Leak	usually; see below	silent	`definitely lost`
Branch on unset heap bytes	silent	silent	`uninitialised value(s)`
`INT_MAX + 1`	silent	`signed integer overflow`	silent
Shift by 32	silent	`shift exponent 32`	silent

Bug	ASan	UBSan	valgrind
Divide by zero	`FPE`, a crash	`division by zero`	`SIGFPE`, a crash

UBSan saw the local array only because the code indexed it by its own name; through a pointer parameter, only ASan spoke. Valgrind reports an unset value when it decides a branch or reaches output, not when it is merely copied; `--track-origins=yes` names the `malloc` it came from. A UBSan report does not stop the program: the overflow toy printed its wrapped value and exited 0. Built with `-fno-sanitize-recover=all`, it stopped at the report with status 1.

Two tools, two opinions

The leaky specimen, built twice with the same flags:

```
$ gcc -Wall -Wextra -Werror -g -fsanitize=address,undefined strip_letter.c -o san_gcc
$ ./san_gcc world z; echo $?
world
0
$ clang -Wall -Wextra -Werror -g -fsanitize=address,undefined strip_letter.c -o san_clang
$ ./san_clang world z 2>&1 | grep SUMMARY
SUMMARY: AddressSanitizer: 6 byte(s) leaked in 1 allocation(s).
```

Valgrind reported those 6 bytes, as did gcc's `-fsanitize=address` alone; on another toy, clang was the silent one. A leak report proves a leak; silence proves nothing.

Two more traps. A sanitizer build that finds a leak exits before `printf`'s buffer reaches a file or a pipe (a leaking toy that printed `done` left `out.txt` empty), so compare outputs from a plain build. And on this machine `cc` is clang 12 without `llvm-symbolizer`, so clang's reports show bare addresses after `WARNING: invalid path to external symbolizer!`; gcc's name file and line. Lend clang binutils' `addr2line`:

```
ASAN_OPTIONS=external_symbolizer_path=/usr/bin/addr2line ./san_clang world z
```

Testing yourself

A subject's examples are exact, every byte and every newline, so test with tools that show bytes.

The recipes here and in later chapters use some shell beyond Shell 00 and 01: `for` and `while` loops, `[ ]`, `case`, `set --`, `shift` and `:`. `man bash` covers them all; `man set`, `man shift` and `man colon` open each one's POSIX page, and `man test` the page for `[ ]`.

`cat -e` shows line ends

`cat -e` prints `$` at the end of every line. Four runs, four stories:

```
$ printf 'moon\n' | cat -e
moon$
$ printf 'moon' | cat -e
moon$ printf 'moon \n' | cat -e
moon $
$ printf '\n' | cat -e
$
```

First: text, then the newline. Second: no newline, so no `$`; the `$` after the word is the shell's next prompt, glued on because the line never ended. Third: a space hiding before the newline. Fourth: an empty line, which is not the same as no output.

got, want and `diff`

Put your output in one file, the expected bytes in another, and let `diff` judge. Here a copy of the specimen with its final `write` of `"\n"` deleted shows what a missing newline looks like:

```
./strip_letter "BLUE MOON" x > got.txt
printf 'BLUE MOON\n' > want.txt
diff -u got.txt want.txt | cat -e
```

```
--- got.txt 2026-10-02 14:50:16.988839000 +0000$
+++ want.txt    2026-10-02 14:50:16.988839000 +0000$
@@ -1 +1 @@$
-BLUE MOON$
\ No newline at end of file$
+BLUE MOON$
```

The decoder: `-` lines come from the first file (yours), `+` lines from the second (expected); `@@ -1 +1 @@` says which lines, and `$` is `cat -e`'s line end. `\ No newline at end of file` belongs to the line above it: your last line never ended. Identical files leave `diff` silent with status 0, so `&& echo SAME` turns silence into a word.

bash offers `diff <(./mine) <(./ref)`, but `sh` here is dash, which answers `Syntax error: "(" unexpected`. Temporary files work in every shell.

When two lines look alike and still differ, ask for bytes. `cmp` names the first difference; `od -c` prints every byte, `\n`, `\t` and `\0` spelled out, offsets in octal (`0000011` is 9):

```
$ cmp got.txt want.txt
cmp: EOF on got.txt after byte 9, in line 1
$ od -c got.txt
0000000   B   L   U   E       M   O   O   N
0000011
```

Exit status

A status is part of behavior. `$?` holds the last command's status (in a pipeline, the pipe's last command), so test without a pipe. The repaired specimen returns 1 on a wrong count:

```
$ ./strip_letter; echo $?
1
$ ./strip_letter abc > /dev/null || echo FAIL
FAIL
```

Reference tools

A reference tool is an existing program whose output you trust; build `want.txt` with it (Appendix F lists them). Two recipes on invented tasks of later chapters, both printing `SAME` here: `tr` checks `digit_halve` (Characters and ASCII), `numfmt` checks `parse_size` through its `./sizes` driver (Numbers, parsing and overflow in int).

```
./digit_halve "route 66, gate 209" > got.txt
printf '%s\n' "route 66, gate 209" | tr '0123456789' '0011223344' > want.txt
diff -u got.txt want.txt && echo SAME
```

```
for a in 0 512 64K 3M 1G 2147483647 2G 2097152K; do
    n=$(numfmt --from=iec "$a")
    if [ "$n" -gt 2147483647 ]; then
        n=invalid
    fi
    printf '%s -> %s\n' "$a" "$n"
done > want.txt
./sizes 0 512 64K 3M 1G 2147483647 2G 2097152K > got.txt
diff -u got.txt want.txt && echo SAME
```

`numfmt` judges the arithmetic only; malformed text such as `12KB` goes in your own table.

A harness for a function

Many C subjects ask for a function, not a program, and its test `main` lives in a file of its own (Compiling with both compilers). Make that `main` print each input and result in brackets, so empty strings and stray spaces show.

```
#include <stdio.h>

char    *my_function(char *s);

int main(int argc, char **argv)
{
    int i;

    i = 1;
    while (i < argc)
    {
        printf("[%s] -> [%s]\n", argv[i], my_function(argv[i]));
        i++;
    }
    return (0);
}
```

Until the real function exists, `stand_in.c` lets the harness link:

```
char    *my_function(char *s)
{
    return (s);
}
```

```
$ cc -Wall -Wextra -Werror main.c stand_in.c -o try
$ ./try "" hello "two words"
[] -> []
[hello] -> [hello]
[two words] -> [two words]
```

Never hand NULL to `%s`. A stand-in returning NULL printed `(null)` here, but the same NULL in `printf("%s\n", p)` crashed the gcc build (exit 139): gcc had turned that call into `puts` (`nm -u` shows `U puts`). Test for NULL before printing.

A table first, then a loop

First write down what each input must produce, and why; seed the table from Appendix B and grow it from your mistakes. For the specimen:

Input	Expected	Why
`""` and `z`	`$`	empty input still owes the newline
`zzzz` and `z`	`$`	everything removed
`world` and `z`	`world$`	nothing removed fills the buffer
no arguments	nothing, status 1	the guard

Then stop typing tests one at a time. This loop runs the repaired specimen on awkward inputs:

```
for a in "" " " "a" "hello world" "-2147483648" "zzzzzzzzzzzzzzzzzzzzzzzzzzzzzzz"; do
    printf '[%s] -> ' "$a"
    ./strip_letter "$a" z | cat -e
done
```

```
[] -> $
[ ] ->  $
[a] -> a$
[hello world] -> hello world$
[-2147483648] -> -2147483648$
[zzzzzzzzzzzzzzzzzzzzzzzzzzzzzzz] -> $
```

Every run passes two arguments (`""` is still one), so add the wrong counts by hand: none, one, three. A clean `$` does not prove clean memory: a copy of the specimen without line 28, which writes the terminator, printed `abc$` under `cat -e`, because the byte after its copy happened to be zero. So wrap the same loop around valgrind, whose `--error-exitcode` counts invalid reads and uninitialised values as well as leaks. It printed `clean:` four times on the repaired build, and `PROBLEM:` four times both on the one without the `free` and on the one without the terminator:

```
for a in "" "abc" "hello world" "zzzz"; do
    if valgrind -q --leak-check=full --error-exitcode=99 ./strip_letter "$a" z > /dev/null 2>&1
    then
        echo "clean: [$a]"
    else
        echo "PROBLEM: [$a]"
    fi
done
```

Self-check

Q1. `diff -u got.txt want.txt` prints `\ No newline at end of file` right under a `-` line. Whose output is missing what?

Answer: end of the chapter

Q2. Your program prints exactly the subject's example. What has that proved?

Answer: end of the chapter

Self-check answers

> **Answer to Q1**
>
> Yours: `-` lines come from the first file, `got.txt`, and the marker belongs to the line above it. Your last line never got its `\n`.

> **Answer to Q2**
>
> That it handles that one input. Empty strings, missing arguments, edge values and leaks are each a separate run, which is what the table and the loops are for.

Vim survival minimum

vim runs in any terminal, no mouse needed, so it is worth knowing even if you prefer another editor. It has modes: in normal mode keys are commands, in insert mode they are text, and command-line mode takes lines that start with `:`. `i` enters insert mode and `Esc` returns to normal — when in doubt, press `Esc`.

Twelve commands cover a day of editing (each checked on vim 8.2 here):

Keys	What it does
`i`	insert text before the cursor
`Esc`	back to normal mode
`:w`	save
`:q`	quit
`:wq`	save and quit
`:q!`	quit and discard changes
`dd`	delete (cut) the current line

Keys	What it does
`yy`	copy the current line
`p`	paste the cut or copied line below
`u`	undo
`Ctrl-r`	redo
`/word`	search; `n` jumps to the next hit

Type `:set number` first: compilers and debuggers talk in line numbers.

If vim or your terminal dies mid-edit, the unsaved text survives in a hidden swap file beside the file, `.work.c.swp` for `work.c`. Reopening `work.c` shows a warning (trimmed):

```
E325: ATTENTION
Found a swap file by the name ".work.c.swp"
[O]pen Read-Only, (E)dit anyway, (R)ecover, (D)elete it, (Q)uit, (A)bort:
```

Press `q`, run `vim -r work.c` to get the unsaved state back, check it, save with `:w` and quit. The swap file stays behind; `rm .work.c.swp` stops the warning. `vim -r` alone lists the swap files it finds. Rehearse once on a scratch file: type a line, stop typing for five seconds, run `pkill -9 -x vim` from a second terminal, and recover. Vim creates the swap file when it opens the file, but copies your typing into it only once you have been idle for `updatetime` milliseconds (`:set updatetime?` says 4000) or have typed `updatecount` characters (`:set updatecount?` says 200), so a kill sooner than either recovers the file without the line you typed.

`nano` is installed too (GNU nano 6.2; `nano -l` shows line numbers). Any editor that shows line numbers is enough.

Practicing from a blank file

Reading code you understand feels like knowing it, but producing it from nothing is a different skill, and the one you need. So the practice loop is retrieval:

1. Read a chapter, its worked example included.
2. Close the guide.
3. Write the chapter's practice task from an empty file.
4. Compile it with both compilers; test it (Testing yourself) until it passes your own table.
5. Only then reopen the chapter and compare.

What you had to look up in step 5 is what to practice next.

Rebuild what you finished

Some days after finishing an exercise of your own, rebuild it from a blank file without looking at the first version, and let your earlier tests judge. Keep the inputs in `cases.txt`, one argument per line, and the outputs you checked in `want.txt` (here `prog` was a copy of `digit_halve`, and the run printed `SAME`):

```
while IFS= read -r a; do
    ./prog "$a"
done < cases.txt > got.txt
diff -u got.txt want.txt && echo SAME
```

`IFS=` keeps leading spaces and `-r` keeps backslashes, so each line arrives exactly as written; an empty line becomes an empty argument. Fill `want.txt` only from output you verified by hand or with a reference tool: a wrong line there makes every later version wrong the same way.

Invented twins

Change one rule of a task you finished and write the new task cold, such as the Sunday-start calendar of Control flow or `starts_with` in Strings. A twin keeps the skill but not the memory of the text, so you rebuild the reasoning, not the letters.

A mistakes log and old code

Write one line per bug you meet: the input, what happened, the rule you learned. "`./prog ""` crashed — an empty argument is still an argument." Each line becomes a row of your edge-case table, which then runs on everything you write. And some days later, read your own code and explain each line aloud: a line you cannot explain is the next thing to practice, even if it passes its tests.

A drill: the grid walk

Walking a grid without stepping outside it is a skill of its own. Draw a small grid on paper, store it as an array of strings, and write a program that prints the coordinates of every neighbor of a chosen cell. Decide first whether diagonal cells count, and write that rule at the top of the file. Then make it survive corner cells, edge cells and a one-by-one grid.

Part II — Foundations

The first modules' ideas sit under every later chapter: bytes out, loops, characters, addresses and strings.

Output and write

C 00 is where output starts, and every later module prints through what it teaches.

Concepts

`write` and exact bytes. `write(fd, address, count)` sends `count` bytes, starting at `address`, to the open file `fd`: 0 is standard input, 1 standard output, 2 standard error. It wants an address: to print a `char c`, pass `&c`, never `c`. A subject's examples are exact to the byte, newlines included, and `cat -e` shows them (Testing yourself).

Two streams and a return value. Two streams let you redirect output and diagnostics separately: `2>/dev/null` hides fd 2 alone. Which stream a line the subject names belongs on is the subject's call. `write` returns the number of bytes written, or -1 on error:

```
#include <unistd.h>

int main(void)
{
    write(2, "note\n", 5);
    if (write(1, "data\n", 5) != 5)
        return (1);
    return (0);
}
```

```
$ ./streams
note
data
$ ./streams 2>/dev/null
data
$ ./streams > /dev/full; echo $?
note
1
```

`/dev/full` refuses every write, so the second `write` returned -1 and the status said so.

Buffering. `printf` collects bytes in a buffer inside your process and calls `write` later; `write` goes out at once. Mixed on one stream, they reorder:

```
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("a");
    write(1, "b", 1);
    printf("c\n");
    return (0);
}
```

It printed `bac`, to a terminal and through a pipe alike. Use one of them per stream; only this demo mixes them.

Counting system calls. Each `write` is a system call, a trip into the kernel, the part of the system that owns files and devices. `strace` is not installed here, so let gdb count. Save this as `count.gdb`:

```
catch syscall write
commands
silent
continue
end
run > /dev/null
info breakpoints
```

Then run `gdb -q -batch -x count.gdb ./prog 2>&1 | grep 'already hit'`. The catchpoint fires on the way into the kernel and on the way out, so hits ÷ 2 = calls.

Worked example: put_bar

Spec: `void put_bar(int done, int total, int width)` draws a progress bar in one `write`: `[`, then `done * width / total` bytes `#`, then `.` up to `width` bytes, then `]` and a newline. Valid calls have `1 <= total <= 1000000`, `0 <= done <= total` and `1 <= width <= 100`, which keeps `done * width` at most 10^8, inside `int`; any other call writes nothing.

```
#include <unistd.h>

void    put_bar(int done, int total, int width)
{
    char    buf[103];
    int     filled;
    int     i;

    if (total < 1 || total > 1000000 || width < 1 || width > 100)
        return ;
    if (done < 0 || done > total)
        return ;
    filled = done * width / total;
    buf[0] = '[';
    i = 0;
    while (i < width)
    {
        buf[i + 1] = '.';
        if (i < filled)
            buf[i + 1] = '#';
        i++;
    }
    buf[width + 1] = ']';
    buf[width + 2] = '\n';
    write(1, buf, width + 3);
}

int main(void)
{
    put_bar(0, 10, 10);
    put_bar(3, 10, 10);
    put_bar(10, 10, 10);
    put_bar(1, 3, 20);
    put_bar(999999, 1000000, 40);
    put_bar(5, 4, 10);
    return (0);
}



$ ./bar | cat -e
[..........]$
[###.......]$
[##########]$
[######..............]$
[######################################.]$
$ gdb -q -batch -x count.gdb ./bar 2>&1 | grep 'already hit'
    catchpoint already hit 10 times
```

The guards come first, so the invalid `(5, 4, 10)` returned before touching the buffer: six calls, five lines. `buf` holds the most a valid call builds, 100 cells plus `[`, `]` and `\n`. Integer division truncates (1 of 3 on 20 cells fills 6; 999999 of 1000000 on 40 fills 39), so the bar is full only when `done` equals `total`. `write` gets `width + 3`, the bytes actually built; `sizeof(buf)` would send unset bytes.

Ten hits, five calls: one per bar. Sending `buf` one byte at a time printed the same bytes in 105 calls (210 hits). One call per line costs a buffer you must bound; one call per byte costs a kernel trip per byte.

Questions to ask

- Which stream does each byte belong on, and what ends the subject's example: a newline or not?
- Am I giving `write` an address, and a count of bytes that exist there?
- What should my program do when `write` returns less than I asked for?
- How many system calls does one line of output cost, and does it matter here?

Self-check

Q1. Your output looks right on screen, but `cat -e` shows no `$` at the end. What is missing?

Answer: end of the chapter

Q2. `s` points to a string. What is wrong with `write(1, s[i], 1)`?

Answer: end of the chapter

Q3. Without `printf`, how do you print the digit seven?

Answer: end of the chapter

Pitfalls

Input or situation	Classic bug	Defense
Output looks right on screen	Missing final newline	`cat -e`: the last line ends in `$`
Printing one character	`write(1, c, 1)`: a value as an address	Pass `&c`; `-Werror` refuses the value
`printf` and `write` on stdout	Bytes out of order	One of them per stream
A message the subject did not ask for	It lands among the bytes you compare with the subject's example	Leave it out, or send it to fd 2

Practice and check

Practice: put_centered. Write `void put_centered(char *s, int width)`: it writes `s` centered in a field of `width` bytes padded with `.`, the odd pad byte on the right, then `\n`, all in one `write`. If `s` is longer than `width`, it writes `s` and `\n`. `width` and the length of `s` are 0 to 80. Expected: `("abc", 8)` gives `..abc...`, `("ab", 6)` gives `..ab..`, `("", 3)` gives `...`, `("toolong", 4)` gives `toolong`, `("x", 1)` gives `x`, `("abc", 0)` gives `abc`.

With a driver `./centered` making those six calls in order, check the bytes and the call count (twelve hits):

```
./centered > got.txt
printf '%s\n' ..abc... ..ab.. ... toolong x abc > want.txt
diff -u got.txt want.txt && echo SAME
gdb -q -batch -x count.gdb ./centered 2>&1 | grep 'already hit'
```

Self-check answers

> **Answer to Q1**
>
> The final newline. `$` marks a line end, so no `$` means no `\n` was written.

> **Answer to Q2**
>
> It passes the character's value where `write` wants an address. gcc and clang warn even without flags (`-Wint-conversion`), so `-Werror` refuses it. Built anyway, with `s` pointing to `moon` , it does not crash: gdb's `catch syscall write` shows `buf=0x6d` , the byte `m` (109) used as an address, and `write` returns -1 (`Bad address`). Not one letter is printed. Pass `&s[i]` .

> **Answer to Q3**
>
> Store `'0' + 7` in a `char` (the digit characters are contiguous) and pass its address to `write` with a count of 1.

Control flow

C 00 is where loops and conditions arrive; every later module is built from them.

Concepts

Bounds and guards. A `while` condition is the loop's bound. `&&` and `||` stop at the first operand that decides them, so `i < size && tab[i] > 0` reads `tab[i]` only when `i` is in range. In an `else if` chain the first true test wins: narrow cases first.

The Norm. norminette 3.3.58 accepts `while` , `if` , `else` , `return` , `break` and `continue` ; it rejects `for` , `switch` and `case` (FORBIDDEN_CS), `goto` (GOTO_FBIDDEN) and `?:` (TERNARY_FBIDDEN). A body holds at most 25 lines, so split off helpers with one job and a name.

Exits, fenceposts, invariants. An early `return` leaves the body to the normal case; a single exit path writes a final newline once. Seven cells need six separators, as seven fence posts hold six rails; counting one too many is a fencepost error. An invariant is true at the top of every pass (" `day` is the next day to print").

A loop that never ends. Under `gdb` , Ctrl-C stops it, `bt` names the line, and `print` shows which counter stopped moving.

Worked example: print_month

Spec: `void print_month(int first, int days)` prints a month as a grid; `first` (0 = Monday ... 6 = Sunday) is day 1's column and `days` is 28 to 31. Under the header `Mo Tu We Th Fr Sa Su` , one line per week, ending in `\n` : cells two characters wide, right-aligned, one space apart, blanks before day 1, no trailing space.

```
#include <stdio.h>
#include <stdlib.h>

static void show_cell(int day, int col)
{
    if (col > 0)
        printf(" ");
    if (day < 1)
        printf("  ");
    else
        printf("%2d", day);
}

void    print_month(int first, int days)
{
    int day;
    int col;

    printf("Mo Tu We Th Fr Sa Su\n");
    day = 1 - first;
    while (day <= days)
    {
        col = (first + day - 1) % 7;
        show_cell(day, col);
        if (col == 6 || day == days)
            printf("\n");
        day++;
    }
}

int main(int argc, char **argv)
{
    if (argc == 3)
        print_month(atoi(argv[1]), atoi(argv[2]));
    return (0);
}
```

```
$ ./month 6 31 | cat -e
Mo Tu We Th Fr Sa Su$
                   1$
 2  3  4  5  6  7  8$
 9 10 11 12 13 14 15$
16 17 18 19 20 21 22$
23 24 25 26 27 28 29$
30 31$
```

One counter drives the grid: `day` starts at `1 - first`, so the first passes print blanks, and `(first + day - 1) % 7` is each cell's column. The separator goes before every cell outside column 0, so no line ends with a space. The newline follows column 6 or the last day. `static` keeps `show_cell` private to its file (Headers, Makefiles and libraries shows it in `nm`).

Questions to ask

- What does the loop do on the smallest input: empty, zero, one item?
- Can you say its invariant in one sentence, and does it hold on the first and last pass?
- Does each guard come before the access it protects?

Self-check

Q1. How many passes does `while (s[i] != '\0')` make on `""`?

Answer: end of the chapter

Q2. Why can `while (tab[i] != 0 && i < size)` read `tab[size]`?

Answer: end of the chapter

Q3. Why does `show_cell` print the separator before a cell, not after?

Answer: end of the chapter

Pitfalls

Input or situation	Classic bug	Defense
Last item of a list	A separator after it	Separator before all but the first
Index tested after use	`tab[i] > 0 && i < size`	Guard first
Counter changed in one branch	The loop never ends	Change it on every path

Practice and check

Practice: a Sunday-start calendar. Weeks start on Sunday (header `Su Mo Tu We Th Fr Sa`); `first` keeps its meaning. No calendar program is installed here, so expected output is written by hand. One case for a program `month_su`; add a month starting on Sunday and a 28-day month starting on Saturday:

```
cat > want.txt <<'EOF'
Su Mo Tu We Th Fr Sa
          1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30
EOF
./month_su 2 30 > got.txt
diff -u got.txt want.txt && echo SAME
```

Self-check answers

> **Answer to Q1**
>
> Zero: the first byte is the terminator.

> **Answer to Q2**
>
> `&&` evaluates left to right, so `tab[i]` is read before `i` is checked.

> **Answer to Q3**
>
> "Not in column 0" is one test; "not after a line's last cell" needs two (column 6, or the last day).

Characters and ASCII

C 00 treats characters as numbers; C 02 builds on their bands.

Concepts

Small integers in bands. `'A'` is 65: `man 7 ascii` prints the table, and under `sh`, `printf '%d\n' "'A"` prints `65`. Digits (48–57), capitals (65–90) and lowercase letters (97–122) are contiguous, so `c - '0'` is a digit's value and each capital sits 32 below its lowercase letter; six punctuation bytes (91–96) separate the letter bands. Printable bytes run from space (32) to `~` (126); `'\t'` is 9, `'\n'` 10, and `'\0'` is 0, not `'0'` (48). `printf 'a\tb\n' | od -c` shows each byte: `a \t b \n`.

Signedness. Plain `char` may be signed:

```
#include <stdio.h>

int main(void)
{
    char    c;

    c = (char)200;
    printf("%d %d\n", c, (unsigned char)c);
    return (0);
}
```

Both compilers print `-56 200` on this x86-64 machine: one byte, two readings. The C standard leaves the choice to the platform; use `unsigned char` where a byte is a value or an index.

Worked example: digit_halve

Spec: `digit_halve TEXT` prints TEXT with every decimal digit halved, rounded down, other bytes unchanged, then a newline. With any other number of arguments it writes `usage: digit_halve TEXT` and a newline to standard error and exits with status 1.

```
#include <unistd.h>

int main(int argc, char **argv)
{
    int     i;
    char    c;

    if (argc != 2)
    {
        write(2, "usage: digit_halve TEXT\n", 24);
        return (1);
    }
    i = 0;
    while (argv[1][i] != '\0')
    {
        c = argv[1][i];
        if (c >= '0' && c <= '9')
            c = '0' + (c - '0') / 2;
        write(1, &c, 1);
        i++;
    }
    write(1, "\n", 1);
    return (0);
}


$ ./digit_halve "route 66, gate 209"
route 33, gate 104
$ ./digit_halve; echo $?
usage: digit_halve TEXT
1
```

The guard comes first, so the loop below it can read `argv[1]` without a second thought. The value comes out (`c - '0'`), the arithmetic happens (`/ 2`), and the value goes back in (`'0' +`); halving the code of `'7'` (55) would give 27, a control byte.

Questions to ask

- Which bytes does the task change, and which pass through?
- Is each class you test one band or several, and what sits between them?
- Where does your arithmetic land for each band's first and last member?

Self-check

Q1. Why test uppercase and lowercase as two bands?

Answer: end of the chapter

Q2. `(char)200` prints `-56` with `%d`. Why?

Answer: end of the chapter

Q3. What separates `'\0'`, `0` and `'0'`?

Answer: end of the chapter

Pitfalls

Input or situation	Classic bug	Defense
A band's last member	`'9' + 1` is `':'`	Test each band's edges
Mixed case	`'A'` to `'z'` accepts six punctuation bytes	Two ranges
Bytes above 127	Read as negative	`unsigned char`

Practice and check

Practice: char_census. With exactly one argument, print `letters=L digits=D spaces=S other=O` and a newline, counting ASCII letters, digits, spaces (`' '` and `'\t'` only) and every other byte; otherwise print nothing.

`tr -cd SET` keeps only SET's bytes, `tr -d SET` deletes them, and `wc -c` counts the rest. Compare these two counts with your line, then write the other two:

```
s=$(printf 'Tab\there, 3 spaces: ok?')
./char_census "$s"
printf '%s' "$s" | tr -cd 'a-zA-Z' | wc -c
printf '%s' "$s" | tr -d 'a-zA-Z0-9 \t' | wc -c
```

Testing yourself checks `digit_halve` against `tr` too.

Self-check answers

> **Answer to Q1**
>
> Six punctuation bytes (91 to 96) sit between them.

> **Answer to Q2**
>
> Plain `char` is signed here, so 200 reads as 200 − 256. Through `unsigned char` it reads 200.

> **Answer to Q3**
>
> `'\0'` and `0` are both zero, the byte that ends a string; `'0'` is the digit, 48.

Pointers and arrays

C 01 is where pointers arrive; every later module passes addresses around.

Concepts

Addresses. `&x` is the address of `x`; if `p` holds it, `*p` is `x` itself. `NULL` is the address of nothing, and dereferencing it crashes.

Out-parameters. Arguments are copies, so a function that assigns its parameter changes nothing outside; given an address, it can write through it. One address per result is how a function hands back several, as `tally_signs` below does.

Arrays. `tab[i]` means `*(tab + i)`. A function receives an array as its first element's address, so `sizeof(tab)` is the array's size where it is declared and an address's size inside the function; the count travels as a parameter. In `char **argv`, `*argv` is the first string's address and `**argv` its first byte.

Worked example: tally_signs

Spec: `void tally_signs(int *tab, int size, int *neg, int *pos)` stores in `*neg` how many of `tab[0]` … `tab[size - 1]` are negative and in `*pos` how many are positive; zeros count in neither. With `size` 0 it stores 0 and 0, and `tab` may be NULL.

```
#include <limits.h>
#include <stdio.h>

void    tally_signs(int *tab, int size, int *neg, int *pos)
{
    int i;

    *neg = 0;
    *pos = 0;
    i = 0;
    while (i < size)
    {
        if (tab[i] < 0)
            *neg += 1;
        else if (tab[i] > 0)
            *pos += 1;
        i++;
    }
}

int main(void)
{
    int tab[5];
    int neg;
    int pos;

    tab[0] = 3;
    tab[1] = -1;
    tab[2] = 0;
    tab[3] = -7;
    tab[4] = 2;
    tally_signs(tab, sizeof(tab) / sizeof(tab[0]), &neg, &pos);
    printf("neg=%d pos=%d\n", neg, pos);
    tally_signs(NULL, 0, &neg, &pos);
    printf("neg=%d pos=%d\n", neg, pos);
    tab[0] = INT_MIN;
    tab[1] = INT_MAX;
    tally_signs(tab, 2, &neg, &pos);
    printf("neg=%d pos=%d\n", neg, pos);
    return (0);
}
```

```
$ ./tally
neg=2 pos=2
neg=0 pos=0
neg=1 pos=1
```

Both results start at 0, written through their addresses: the caller's variables are uninitialized. `*neg += 1` changes the caller's `neg`; `neg += 1` would move a local copy of the address. With `size` 0 the loop never reads `tab`.

gdb shows both scopes; built with `-g`, under `gdb -q ./tally` (start-up lines trimmed):

```
(gdb) break tally_signs
Breakpoint 1 at 0x1180: file tally.c, line 8.
(gdb) run
Breakpoint 1, tally_signs (tab=0x7ffe43c8e170, size=5, neg=0x7ffe43c8e168, pos=0x7ffe43c8e16c) at
tally.c:8
8       *neg = 0;
(gdb) print sizeof(tab)
$1 = 8
(gdb) up
#1  0x0000555959e45258 in main () at tally.c:32
32      tally_signs(tab, sizeof(tab) / sizeof(tab[0]), &neg, &pos);
(gdb) print &neg
$2 = (int *) 0x7ffe43c8e168
(gdb) print sizeof(tab)
$3 = 20
```

The callee's `neg` equals the caller's `&neg` (your addresses will differ), and `tab` is 8 bytes in one frame, 20 in the other.

Questions to ask

- Which of the caller's variables must change, and does the function get their addresses?
- How does the function know how many elements there are?
- Does every out-parameter get a value on every path?

Self-check

Q1. Why can't a function change variables it receives as plain values?

Answer: end of the chapter

Q2. Inside a function, `sizeof` on the parameter `int *tab` gives 8. Why?

Answer: end of the chapter

Q3. A caller passes `tab` NULL with `size` 3. What happens, and who broke the contract?

Answer: end of the chapter

Pitfalls

Input or situation	Classic bug	Defense
A result the caller needs	`neg = …`	`*neg = …`
`sizeof` on a parameter	Gives 8	Pass the count
The last index	`i <= size`	Indexes run 0 to `size - 1`

Practice and check

Practice: rotate_left. `int rotate_left(int *values, int count)` moves every element one place left, in place, the first going to the end, and returns the value that moved: `{1, 2, 3, 4}` becomes `{2, 3, 4, 1}` and the call returns 1. A count of 0 changes nothing and returns 0; a count of 1 changes nothing and returns that element.

Two properties check it: one call puts the old `values[0]` in `values[count - 1]` and returns it, and `count` calls restore the original. Built with `-fsanitize=address`, a driver that reads `values[count]` of a local array reports `stack-buffer-overflow`; a plain build is silent.

Self-check answers

> **Answer to Q1**
>
> It receives copies; it needs their addresses to write into the caller's memory.

> **Answer to Q2**
>
> The array arrived as its first element's address, and `sizeof` measures the address.

> **Answer to Q3**
>
> The first pass reads `tab[0]` through NULL, and the program dies of a segfault (exit status 139). The caller broke the contract: the spec allows NULL only with `size` 0, where `i < size` is false before the first pass and `tab` is never read.

Strings

C 02 and C 03 are about strings; every later module passes them around.

Concepts

The terminator. A C string is a `char` array ending in `'\0'`; its length counts the bytes before it, its size is length plus one.

Vacuous truth. A claim about every character of `""` is true: no character can refute it. Your subject says whether it uses that convention.

Literals are read-only. Writing into a string literal compiles and crashes; an array initialized from a literal is your own copy. This deliberately broken demo does both (norminette flags line 5 with DECL_ASSIGN_LINE, as the Norm separates declaration and assignment; the C is valid):

```
#include <unistd.h>

int main(void)
{
    char    copy[] = "abc";
    char    *lit;

    lit = "abc";
    copy[0] = 'A';
    write(1, copy, 3);
    write(1, "\n", 1);
    lit[0] = 'A';
    write(1, lit, 3);
    write(1, "\n", 1);
    return (0);
}


$ ./lit
Abc
Segmentation fault
$ echo $?
139
```

gcc and clang build it without a warning, and both builds crash: 139 is 128 + 11, signal 11 being SIGSEGV.

Contracts live in man pages. Before a bounded loop, ask what a bound of zero means; find the sentence about the terminator in `man 3 strncpy`; read RETURN VALUE, not only DESCRIPTION.

Worked example: ends_with

Spec: `int ends_with(char *s, char *suffix)` returns 1 if the last `strlen(suffix)` bytes of `s` equal `suffix`, else 0; an empty suffix gives 1, one longer than `s` gives 0. It calls `strlen` because the lesson is the offsets; strings stay shorter than `INT_MAX` bytes. The driver brackets each pair of arguments.

```c
#include <stdio.h>
#include <string.h>

int ends_with(char *s, char *suffix)
{
    int len_s;
    int len_suf;
    int i;

    len_s = strlen(s);
    len_suf = strlen(suffix);
    if (len_suf > len_s)
        return (0);
    i = 0;
    while (i < len_suf)
    {
        if (s[len_s - len_suf + i] != suffix[i])
            return (0);
        i++;
    }
    return (1);
}

int main(int argc, char **argv)
{
    int i;

    i = 1;
    while (i + 1 < argc)
    {
        printf("[%s] [%s] %d\n", argv[i], argv[i + 1],
            ends_with(argv[i], argv[i + 1]));
        i += 2;
    }
    return (0);
}
```

```
$ ./ends main.c .c c .c "" "" abc "" "" x aa a
[main.c] [.c] 1
[c] [.c] 0
[] [] 1
[abc] [] 1
[] [x] 0
[aa] [a] 1
```

The guard comes first: for a longer suffix, `len_s - len_suf` is negative, and even forming an address before `s` is undefined. Past it, `len_s - len_suf` is where the tail starts. An empty suffix makes zero passes and returns 1.

Questions to ask

- What does your code do with `""` as each argument in turn?
- Does anything you compute land before a string's start or past its terminator?
- What does the contract say happens at the bound, and what comes back?
- Which bytes separate the words your subject talks about: one character, or a set? Run your code on a quoted tab, and on separators at the start, at the end and in runs.

Self-check

Q1. Is "every character of `""` passes the test" true?

Answer: end of the chapter

Q2. A function bounded by `n` gets `n` equal to 0. What is certain?

Answer: end of the chapter

Q3. For `char s[] = "hello";`, what are `sizeof(s)` and `strlen(s)`?

Answer: end of the chapter

Pitfalls

Input or situation	Classic bug	Defense
`""`	Reading `s[len - 1]`	Run every function on `""`
`" "`	A scan for a non-space runs off the end	Every scan also tests `'\0'`
Length and size	No room for `'\0'`	Size is length + 1

Practice and check

Practice: starts_with. `int starts_with(char *s, char *prefix)` returns 1 if `s` begins with `prefix`, else 0; an empty prefix gives 1, one longer than `s` gives 0. Give it a driver like `ends_with`'s.

The shell's `case` patterns are a reference: `*"$2"` matches a string ending with `$2`, `"$2"*` one starting with it. This loop rebuilds the `ends_with` driver's output:

```
set -- main.c .c c .c "" "" abc "" "" x aa a
./ends "$@" > got.txt
: > want.txt
while [ $# -ge 2 ]; do
  case "$1" in *"$2") r=1 ;; *) r=0 ;; esac
  printf '[%s] [%s] %d\n' "$1" "$2" "$r" >> want.txt
  shift 2
done
diff -u got.txt want.txt && echo SAME
```

For `starts_with`, run your program and match `"$2"*`.

Self-check answers

> **Answer to Q1**
>
> Yes, vacuously. Your subject says whether it follows that convention.

> **Answer to Q2**
>
> Only that zero bytes of the bounded input are compared or copied; whether a terminator is written, and what comes back, is in the contract.

Answer to Q3

6, counting the terminator, and 5.

Part III — Numbers and recursion

Text becomes numbers and back, numbers become bits, and functions call themselves.

Numbers, parsing and overflow in int

C 04 is where text becomes numbers; C 05 and C 07 compute with them.

Concepts

The edges. Here `int` runs from `INT_MIN`, -2147483648, to `INT_MAX`, 2147483647 (`<limits.h>`). Unsigned arithmetic wraps; signed overflow is undefined behavior: C promises nothing about the result, not even a wrong number, so test before it can happen, never after.

`INT_MIN` has no positive twin. Its magnitude is `INT_MAX + 1`, so negating it overflows, on exactly one input. Before any negation, ask which values can reach it, and run `-2147483648` through your code. `long` is no escape: the C standard lets it be as narrow as `int`, and on 32-bit x86 it is (`gcc -m32 -dM -E - < /dev/null | grep __SIZEOF_LONG__` prints `#define __SIZEOF_LONG__ 4`).

Digits. Decimal text is positional: `472` is ((4 × 10) + 7) × 10 + 2, and `n * 10 + d` fits in `int` exactly when `n <= (INT_MAX - d) / 10`. What may surround the digits is each subject's rule. Backwards, `n % 10` is the last digit and `n / 10` drops it, so digits come out reversed; for negative `n`, `/` truncates toward zero and `%` takes `n`'s sign.

Division traps. `/ 0`, `% 0` and `INT_MIN / -1` are undefined. This demo prints `-a`, `a / b` and `a % b`:

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    int a;
    int b;

    if (argc != 3)
        return (1);
    a = atoi(argv[1]);
    b = atoi(argv[2]);
    printf("%d %d %d\n", -a, a / b, a % b);
    return (0);
}
```

```
$ ./quot -7 2
7 -3 -1
$ ./quot 7 0
Floating point exception
$ ./quot -2147483648 -1
Floating point exception
```

Both bad divisions trap (status 136: 128 + 8, SIGFPE). `./quot -2147483648 1` prints `-2147483648 -2147483648 0` silently; a gcc `-fsanitize=undefined` build adds:

```
quot.c:13:2: runtime error: negation of -2147483648 cannot be represented in type 'int'; cast to an
unsigned type to negate this value to itself
```

Number theory as knowledge. Trial division tries candidate divisors in turn; divisors pair up (`a × b = n`), so ask how far it must go (Self-check Q1) and how to stop without overflow near `INT_MAX`. Euclid's algorithm finds a gcd (greatest common divisor) with remainders alone. The identity a × b = gcd × lcm (least common multiple) invites a product that leaves `int`: which order of the same operations stays in range?

Worked example: parse_size

Spec: `int parse_size(char *s, int *bytes)` accepts one or more decimal digits, then optionally `K`, `M` or `G` (× 1024, × 1024^2, × 1024^3), then the end. If the text is valid and the value fits in `int`, it stores the value and returns 1; otherwise it returns 0 and stores nothing. No sign, no whitespace, no lowercase unit.

```
#include <limits.h>
#include <stdio.h>

static int  scale_of(char c)
{
    if (c == 'K')
        return (1024);
    if (c == 'M')
        return (1024 * 1024);
    if (c == 'G')
        return (1024 * 1024 * 1024);
    return (0);
}

int parse_size(char *s, int *bytes)
{
    int n;
    int i;
    int scale;

    n = 0;
    i = 0;
    while (s[i] >= '0' && s[i] <= '9')
    {
        if (n > (INT_MAX - (s[i] - '0')) / 10)
            return (0);
        n = n * 10 + (s[i] - '0');
        i++;
    }
    if (i == 0)
        return (0);
    scale = 1;
    if (s[i] != '\0')
        scale = scale_of(s[i++]);
    if (scale == 0 || s[i] != '\0' || n > INT_MAX / scale)
        return (0);
    *bytes = n * scale;
    return (1);
}

int main(int argc, char **argv)
{
    int i;
    int bytes;

    i = 1;
    while (i < argc)
    {
        if (parse_size(argv[i], &bytes))
            printf("%s -> %d\n", argv[i], bytes);
        else
            printf("%s -> invalid\n", argv[i]);
        i++;
    }
    return (0);
}
```

```
$ ./sizes 64K 1G 2097151K 2G 2097152K 2147483647 2147483648 12KB -1
64K -> 65536
1G -> 1073741824
2097151K -> 2147482624
2G -> invalid
2097152K -> invalid
2147483647 -> 2147483647
2147483648 -> invalid
12KB -> invalid
-1 -> invalid
```

Each digit is tested before it is added, so `2147483648` stops at its last digit. `i == 0` rejects `""`, `K`, `-1` and `" 1"`. Scaling has its own test, which `2097152K` fails. In the last `if`, `scale == 0` (an unknown unit, as in `"1 "`) is tested before `INT_MAX / scale` divides by it. Nothing is stored until every test passes.

Questions to ask

- Which inputs does the subject promise, and does every intermediate value stay in `int` for all of them?
- What may surround the digits, and which sentence of your subject says so?
- Can a divisor be zero, or `INT_MIN` meet -1?

Self-check

Q1. Divisors of `n` pair up. How far must trial division go?

Answer: end of the chapter

Q2. Why can't you negate `INT_MIN`?

Answer: end of the chapter

Q3. State Euclid's gcd algorithm in one sentence.

Answer: end of the chapter

Pitfalls

Input or situation	Classic bug	Defense
`-2147483648`	Negating it	Run that value through your code, under UBSan too
Many digits	Overflow, then test	Test before the step
Whitespace, signs	Another subject's rules	Your subject's rules

Practice and check

Practice: checked_add. `int checked_add(int a, int b, int *sum)` stores `a + b` and returns 1 when the sum fits in `int`, else returns 0 and stores nothing; it uses `int` arithmetic only and never evaluates an overflowing expression. Its driver `./add A B` prints the sum or `overflow`.

`bc` prints the true sum at any size: here 2147483648, too big for `int`, so the driver owes `overflow`. Build it with `-fsanitize=undefined` as well; on the edge cases no `runtime error` line may appear:

```
echo '2147483647 + 1' | bc
./add 2147483647 1
./add -2147483648 -1
```

Testing yourself checks `parse_size` against `numfmt --from=iec`.

Self-check answers

> **Answer to Q1**
>
> To the square root of `n`: a divisor above it pairs with one below, already met. The stopping test must not overflow `int` near `INT_MAX`.

> **Answer to Q2**
>
> Its magnitude, 2147483648, is `INT_MAX + 1`, which no `int` holds: `-INT_MIN` is signed overflow, undefined behavior, as `./quot -2147483648 1` showed under UBSan.

> **Answer to Q3**
>
> Replace the pair with the second number and the remainder of the first divided by the second; when the second is zero, the first is the gcd.

Bases and bits

C 04 is where numbers meet bases other than ten.

Concepts

Positional notation. In base B each place is worth B times the place to its right: `1010` in base two is 8 + 2. Reading digits is the accumulation of Numbers, parsing and overflow in int with B for ten; given digits as a string of symbols, a symbol's value is its index there. Going the other way, `n % B` is the last digit and `n / B` drops it: digits come out least significant first.

Tools. `printf '%x %o\n' 255 255` prints `ff 377`. `bc` prints in the base `obase` names: `echo 'obase=2; 5' | bc` prints `101`, without leading zeros, so compare values, not padded strings.

Bits. `n >> k` moves bits down `k` places, `n << k` up; `&` with a mask keeps chosen bits, `|` merges. `(n >> 3) & 1` reads bit 3, and the shell has the same operators: `echo $(( (85 >> 2) & 1 ))` prints `1`. A hex digit is four bits, so a byte is two, as `od` shows (also try `xxd -b` and `hexdump -C`):

```
$ printf 'Hex\t\200\n' | od -An -tx1
 48 65 78 09 80 0a
```

Signs and edges. That `80` is 128, but a plain `char` on x86-64 reads it as -128 (see Characters and ASCII), so byte work uses `unsigned char`; shifting a negative value right brings in sign bits (`-8 >> 3` printed -1 under both compilers). On a 32-bit `int`, `1 << 31` is undefined, as is a count of 32 or more. Both compilers refuse a constant count of 32 or more under `-Werror`

(`shift count >= width of type`), but neither says a word about `1 << 31`. UBSan reports it at run time: `left shift of 1 by 31 places cannot be represented in type 'int'`. `1u << 31` is defined.

`void *` is an address with no type to read through; to look at the bytes of anything, read them through `unsigned char *`. gdb shows what such a pointer sees: with `int n = 258;`, `x/4xb &n` prints `0x02 0x01 0x00 0x00` under both compilers here, the low byte first, as x86-64 stores an `int`.

Worked example: count_trailing_zeros

Spec: `int count_trailing_zeros(unsigned int n)` returns the number of zero bits below the lowest set bit of a 32-bit unsigned int, defined as 32 for 0.

```
#include <stdio.h>

int count_trailing_zeros(unsigned int n)
{
    int count;

    count = 0;
    while (count < 32 && (n & 1) == 0)
    {
        n >>= 1;
        count++;
    }
    return (count);
}

int main(void)
{
    printf("%d\n", count_trailing_zeros(8));
    printf("%d\n", count_trailing_zeros(160));
    printf("%d\n", count_trailing_zeros(7));
    printf("%d\n", count_trailing_zeros(0));
    return (0);
}
```

It prints `3`, `5`, `0` and `32`. 160 is `10100000` (`echo 'ibase=2; 10100000' | bc` prints `160`); the bits above the lowest set bit do not matter, so 5.

`n & 1` probes the bottom bit; `n >>= 1` moves the next one down. On 0 the probe never stops the loop; the `count < 32` bound does, at the defined answer — a guard built into the bound. The `unsigned` parameter makes every shift bring in zeros.

Questions to ask

- Which end do your digits come out of, and which does your output need first?
- Is the value you shift signed, and what enters at the top when it moves right?
- Does your reference tool drop leading zeros that your output keeps?

Self-check

Q1. Why is a digit set that repeats a symbol, or holds a sign, ambiguous?

Answer: end of the chapter

Q2. Why does code that prints bytes in hex go through `unsigned char`?

Answer: end of the chapter

Q3. What is `(n >> 4) & 15` for `n = 0xA7`?

Answer: end of the chapter

Pitfalls

Input or situation	Classic bug	Defense
The value 0	`while (n > 0)` peels no digit	Run 0 itself
A negative value shifted right	A shift-until-zero loop never ends	Shift `unsigned` values
Bit 31 of an `int`	`1 << 31` is undefined	`1u << 31`; UBSan on the edges

Practice and check

Practice: popcount. Write `int popcount(unsigned int n)`, returning the number of set bits, and a driver `./popcount N...` printing `N COUNT` per argument. `bc` and `grep` build the expected file:

```
for n in 0 1 85 2147483648 4294967295; do
    echo "$n $(echo "obase=2; $n" | bc | grep -o 1 | wc -l)"
done > want.txt
./popcount 0 1 85 2147483648 4294967295 > got.txt
diff -u got.txt want.txt && echo SAME
```

Self-check answers

> **Answer to Q1**
>
> A symbol's value is its index in the set: a repeated symbol has two, and a sign in the set looks like the number's own sign. What a program does with such a set is its subject's rule.

> **Answer to Q2**
>
> Plain `char` is signed on x86-64: bytes above 127 turn negative, and their hex comes out wrong.

> **Answer to Q3**
>
> 10: the high hex digit, `a`.

Recursion

C 05 is where functions call themselves; C 13's trees lean on it.

Concepts

Base case first. A recursive function answers the smallest input directly and any other by calling itself on a smaller one; say the base case in words, then check every input reaches it.

Frames. Each call has its own parameters and locals, stacked on a finite stack (`ulimit -s` prints `8192` , in KB). Recursing on `n - 2` toward a base case `n == 0` crashes on 7 — `Segmentation fault` , exit status 139 — as 5, 3, 1, -1 skip 0; in one run, gdb's `bt -2` put `main` at frame #261849.

Trust the smaller call. Assume it works, and check only that this call builds its answer from it. **Backtracking** is recursion over choices: mark one, recurse for the rest, unmark it before the next, so every branch starts from the same state.

Worked example: digital_root

Spec: `digital_root DIGITS` prints the digital root of DIGITS (sum the digits, then the sum's digits, until one digit remains) and `\n` . An empty or non-digit argument prints `Error` and `\n` on standard output, exit status 1. Any argument count other than one writes `usage: digital_root DIGITS` and a newline to standard error and exits 2.

```c
#include <unistd.h>

int digit_sum(int n)
{
    int sum;

    sum = 0;
    while (n > 0)
    {
        sum = sum + (n % 10);
        n = n / 10;
    }
    return (sum);
}

int digital_root(int n)
{
    if (n < 10)
        return (n);
    return (digital_root(digit_sum(n)));
}

int sum_digits_of(char *s, int *sum)
{
    int i;

    *sum = 0;
    i = 0;
    while (s[i] != '\0')
    {
        if (s[i] < '0' || s[i] > '9')
            return (0);
        *sum = *sum + (s[i] - '0');
        i++;
    }
    return (i > 0);
}

int main(int argc, char **argv)
{
    int     sum;
    char    digit;

    if (argc != 2)
    {
        write(2, "usage: digital_root DIGITS\n", 27);
        return (2);
    }
    if (sum_digits_of(argv[1], &sum) == 0)
    {
        write(1, "Error\n", 6);
        return (1);
    }
    digit = (char)('0' + digital_root(sum));
    write(1, &digit, 1);
    write(1, "\n", 1);
    return (0);
}
```

```
$ ./digital_root 9875 | cat -e
2$
$ ./digital_root "" | cat -e
Error$
$ ./digital_root 1 2; echo $?
usage: digital_root DIGITS
2
```

`digital_root` answers a one-digit number itself and calls itself on the digit sum of any other, a smaller number: 9875 sums to 29, then 11, then 2. `sum_digits_of` validates each byte before use and keeps `main` within the Norm's 25 lines. Each digit adds at most 9, and `getconf ARG_MAX` caps a command line at 2097152 bytes here, so the sum fits `int`.

Worked example: hanoi

Spec: move `n` disks from peg `A` to peg `C` one at a time, never a larger disk onto a smaller, printing each move as `disk K: X -> Y`. `./hanoi N` accepts `0 <= N <= 20`, else prints `usage: hanoi N` on stderr and exits 2. Its test driver uses libc `atoi` (`abc` reads as 0).

```
#include <stdio.h>
#include <stdlib.h>

void    hanoi(int n, char from, char to, char via)
{
    if (n == 0)
        return ;
    hanoi(n - 1, from, via, to);
    printf("disk %d: %c -> %c\n", n, from, to);
    hanoi(n - 1, via, to, from);
}

int main(int argc, char **argv)
{
    int n;

    n = -1;
    if (argc == 2)
        n = atoi(argv[1]);
    if (n < 0 || n > 20)
    {
        fprintf(stderr, "usage: hanoi N\n");
        return (2);
    }
    hanoi(n, 'A', 'C', 'B');
    return (0);
}
```

```
$ ./hanoi 2
disk 1: A -> B
disk 2: A -> C
disk 1: B -> C
```

By the trust rule, moving `n` disks is: the `n - 1` above the largest onto `via`, disk `n` to `to`, the `n - 1` back on top. A conditional breakpoint stops on the smallest calls (addresses hidden, other replies trimmed):

```
(gdb) break hanoi if n == 1
(gdb) run 3 > /dev/null
(gdb) set print address off
(gdb) bt
#0  hanoi (n=1, from=65 'A', to=67 'C', via=66 'B') at hanoi.c:6
#1  hanoi (n=2, from=65 'A', to=66 'B', via=67 'C') at hanoi.c:8
#2  hanoi (n=3, from=65 'A', to=67 'C', via=66 'B') at hanoi.c:8
#3  main (argc=2, argv=) at hanoi.c:25
```

Each frame holds its own `n`, `from` and `to`; the stack never holds more than `n + 1` of them, yet each disk doubles the moves: `./hanoi 10 | wc -l` prints `1023`, as does `echo '2^10 - 1' | bc`.

Questions to ask

- What is the smallest input, and what is its answer with no further call?
- Does every call move closer to the base case, negative and odd inputs included?
- How deep does your largest input go, and how many calls does it make?

Self-check

Q1. `./hanoi 20` prints 1048575 moves. Why never more than 21 frames of `hanoi`?

Answer: end of the chapter

Q2. In backtracking, why undo only after the recursive call returns?

Answer: end of the chapter

Q3. What does gdb's `bt` show that one `print` cannot?

Answer: end of the chapter

Pitfalls

Input or situation	Classic bug	Defense
A step that skips the base case	Endless calls; exit status 139	Check every input reaches it
Two self-calls per frame	Exponential work looks like a hang	Count calls against a formula
Backtracking	A choice sees the last one's mark	Undo what you did, after the call

Practice and check

Practice: print_partitions. Write `void print_partitions(int n)` (`n` from 1 to 30), printing every way to write `n` as a sum of positive parts in non-increasing order: one space between parts, one sum per line, largest first part first. For 4: `4`, `3 1`, `2 2`, `2 1 1`, `1 1 1 1`. With a driver `./partitions N`, these print 42, then nothing (every line adds up):

```
./partitions 10 | wc -l
./partitions 10 | awk '{ s = 0; for (i = 1; i <= NF; i++) s += $i; if (s != 10) print }'
```

Self-check answers

> **Answer to Q1**
>
> A call finishes its first self-call before the second, so the stack holds one frame per value of `n`, 20 to 0: depth is not call count.

> **Answer to Q2**
>
> The call explores everything that follows from the mark; the mark must stand until it returns.

> **Answer to Q3**
>
> Every frame on the stack, each with its own parameters: the chain of calls that led here.

Part IV — Programs, memory and builds

This part covers whole programs, the memory they own, and builds of more than one file.

Programs and argv

C 06 is where `main` takes arguments.

Concepts

The layout. `argv[0]` is the program's name, `argv[1]` to `argv[argc - 1]` the arguments, and `argv[argc]` is NULL, so with no arguments `argv[1]` is NULL, and using it as a string crashes. gdb shows the array's end (replies to the first two commands trimmed):

```
(gdb) break main
(gdb) run "a b" c
(gdb) print argv[argc]
$1 = 0x0
(gdb) print argv[1]@argc
$2 = {0x7ffff19bb166 "a b", 0x7ffff19bb16a "c", 0x0}
```

The shell builds argv. Quotes group words and never reach your program: `set -- "" "a b" c; echo $#` prints `3` under `sh`.

Exit status and stderr. `main`'s return value is the exit status, which Testing yourself reads with `echo $?: ./digital_root 12a; echo $?` prints `Error` then `1`. `hanoi`'s spec sends its usage line to stderr, so `./hanoi 2>/dev/null; echo $?` prints only `2`; `digital_root`'s spec puts `Error` on standard output. Each spec says which stream a message uses.

Program or function. The subject says which; a function's file holds no `main` (Compiling with both compilers).

Worked example: initials

Spec: `initials WORD...` prints the first byte of each non-empty argument, in order, then one `\n`. With no arguments it writes `usage: initials WORD...` and a newline to standard error and exits 1.

```
#include <unistd.h>

int main(int argc, char **argv)
{
    int i;

    if (argc < 2)
    {
        write(2, "usage: initials WORD...\n", 24);
        return (1);
    }
    i = 1;
    while (i < argc)
    {
        if (argv[i][0] != '\0')
            write(1, &argv[i][0], 1);
        i++;
    }
    write(1, "\n", 1);
    return (0);
}
```

```
$ ./initials portable network graphics | cat -e
png$
$ ./initials "" zebra | cat -e
z$
$ ./initials; echo $?
usage: initials WORD...
1
```

The guard turns away a command line with no words before the loop starts, so every run that reaches the loop has at least one argument to read. The loop starts past the program's name and runs while `i < argc`. An empty argument's first byte is the terminator, so the test skips it.

Questions to ask

- How many arguments does your subject expect, and what must print for any other count?
- What does your program do with `""`, and with an argument that holds a space?
- Which stream does each message use, and which exit status does each path return?

Self-check

Q1. `./prog` with no arguments segfaults. What do you check first?

Answer: end of the chapter

Q2. What is `argc` for `./prog "a b" c`?

Answer: end of the chapter

Q3. Can a program reuse another subject's output for a missing argument?

Answer: end of the chapter

Pitfalls

Input or situation	Classic bug	Defense
`./prog` alone	Using `argv[1]` as a string while it is NULL	Check `argc` first
`./prog ""`	`argc` is 2, but the argument has no bytes	Test it on purpose
`./prog "a b"`	Expecting two arguments	The shell decides the words; quote in tests

Practice and check

Practice: longest. Write a program printing its longest argument (the first on a tie) and its length as `ARG (N)` and `\n`; an empty argument has length 0; no arguments, no output. These should print `coffee (6)$`, `ab (2)$`, `(0)$` and `0`:

```
./longest tea coffee milk | cat -e
./longest ab cd | cat -e
./longest "" | cat -e
./longest | wc -c
```

Self-check answers

> **Answer to Q1**
>
> Whether anything uses `argv[1]` as a string before checking `argc`; with no arguments it is the NULL ending the array.

> **Answer to Q2**
>
> 3: the program's name and two arguments. The quotes made `a b` one word.

> **Answer to Q3**
>
> Only if its own subject says so: each subject defines that output; never copy one from another.

The heap and ownership

C 07 is where memory starts coming from `malloc`.

Concepts

Lifetimes. A local lives until its function returns, a `malloc` block until it is freed. Returning a local array is refused under `-Werror`: gcc says `function returns address of local variable`, and clang's `-Wreturn-stack-address` agrees.

Size, allocate, check. Count before calling `malloc` — a string needs its length plus 1 for the terminator, `n` ints `sizeof(int) * n` — check that the size fits in `int` before you multiply, and check every result against NULL.

Ownership. A function returning a block hands it to its caller; the owner frees it once — free what you built, where your subject allows `free`. A freed pointer keeps the old address; this demo reads through one, and with an argument frees it twice:

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    int *box;

    (void)argv;
    box = malloc(sizeof(int));
    if (box == NULL)
        return (1);
    *box = 42;
    free(box);
    if (argc > 1)
        free(box);
    printf("%d\n", *box);
    return (0);
}
```

Built with `gcc -g -fsanitize=address`, `./uaf` reports `heap-use-after-free` at `uaf.c:16`, freed at line 13, allocated at line 9; `./uaf x` stops at line 15 with `attempting double-free`.

Arrays of pointers. argv is the model: `argc + 1` pointers, the last one NULL. A heap-built array of strings has that shape: a block of pointers plus a block per string. Before the first `malloc`, ask how many pointers it needs, and how you can know that number.

Worked example: rle_encode

Spec: return a new string where each run of one repeated character becomes the character and the run's length as one digit, a run over 9 becoming several: `"aaabcc"` becomes `"a3b1c2"`. The result is at most twice the input's length: its size fits `int` for inputs under a billion bytes.

```c
#include <stdlib.h>
#include <unistd.h>

int encoded_size(char *s)
{
    int size;
    int run;
    int i;

    size = 0;
    i = 0;
    while (s[i] != '\0')
    {
        run = 1;
        while (run < 9 && s[i + run] == s[i])
            run++;
        size = size + 2;
        i = i + run;
    }
    return (size);
}

char    *rle_encode(char *s)
{
    char    *out;
    int     i;
    int     pos;
    int     run;

    out = malloc(encoded_size(s) + 1);
    if (out == NULL)
        return (NULL);
    i = 0;
    pos = 0;
    while (s[i] != '\0')
    {
        run = 1;
        while (run < 9 && s[i + run] == s[i])
            run++;
        out[pos] = s[i];
        out[pos + 1] = (char)('0' + run);
        pos = pos + 2;
        i = i + run;
    }
    out[pos] = '\0';
    return (out);
}

int main(int argc, char **argv)
{
    char    *encoded;

    if (argc != 2)
        return (1);
    encoded = rle_encode(argv[1]);
    if (encoded == NULL)
        return (1);
    write(1, "[", 1);
    write(1, encoded, encoded_size(argv[1]));
    write(1, "]\n", 2);
    free(encoded);
```

```
    return (0);
}
```

```
$ ./rle_demo "aaaaaaaaaaaab" | cat -e
[a9a3b1]$
$ ./rle_demo "" | cat -e
[]$
```

`valgrind --leak-check=full ./rle_demo aaabcc` reports `1 allocs, 1 frees, 7 bytes allocated` (six for `a3b1c2`, one for the terminator) and `no leaks are possible`.

`encoded_size` measures, two bytes per run, and allocates nothing; `rle_encode` repeats its run detection line for line, so the fill writes exactly what the measurement paid for. The cap of 9 keeps each count one digit, which `'0' + run` converts; the `+ 1` buys the terminator; `main` frees the result.

Questions to ask

- How many bytes does the result need, terminator included, counted before you allocate?
- Who owns the block once your function returns, and does your subject allow `free`?
- If an allocation fails halfway, what is already built, and what does your subject return?

Self-check

Q1. Why does copying a string of length ten need eleven bytes?

Answer: end of the chapter

Q2. Who frees a malloc'd array a function returns, and what is the function's duty?

Answer: end of the chapter

Q3. Why must `encoded_size` and `rle_encode` detect runs by the same rule?

Answer: end of the chapter

Pitfalls

Input or situation	Classic bug	Defense
A copy of a 10-byte string	`malloc(len)`: no room for `'\0'`	Allocate `len + 1`
Failure on the third of eight blocks	The first two leak	Free them where `free` is allowed
An array of strings	Its end cannot be found	A slot for the sentinel when the subject expects one

Practice and check

Practice: repeat_str. Write `char *repeat_str(char *s, int times)`: a new string holding `s` `times` times, empty for 0, or NULL when `times` is negative, the size would not fit in `int`, or `malloc` fails. With a driver `./repeat S N` printing the result and `\n`, the shell's `printf`, which reuses its format per argument, builds the expected bytes:

```
./repeat ab 3 > got.txt
{ printf 'ab%.0s' $(seq 3); echo; } > want.txt
diff -u got.txt want.txt && echo SAME
```

`printf` with no arguments still prints its format once, so write `times` 0 by hand; run every case under `valgrind --leak-check=full`.

Self-check answers

> **Answer to Q1**
>
> Length counts the bytes before the terminator; the copy needs its own. A ten-byte block overflows, which a plain run often hides and a sanitizer reports.

> **Answer to Q2**
>
> The caller: ownership moves with the return. The function's duty is its failure paths: check every `malloc`, and on an early exit free what it built, where its subject allows `free`.

> **Answer to Q3**
>
> `encoded_size` sizes the block and `rle_encode` writes into it; if they disagree on one input, the fill overruns.

Headers, Makefiles and libraries

C 08 is where headers and macros arrive; C 09 and C 10 add libraries and Makefiles.

Concepts

Declarations and the link. The compiler sees one `.c` file at a time; the linker then matches calls with definitions by name alone. A prototype retyped by hand can disagree with the definition and still link — then run wrong. So copy prototypes from your subject character for character, and declare each function once, in a header that the caller and the defining file both include.

`#include` and the guard. The preprocessor pastes the named file in place of the line: `<stdio.h>` from the system's directories, `"size_parse.h"` from beside the including file first, and `-I DIR` adds a directory. A guard makes a second paste into the same file harmless; norminette wants it named after the file (HEADER_PROT_NAME):

```
#ifndef SIZE_PARSE_H
# define SIZE_PARSE_H

int parse_size(char *s, int *bytes);

#endif
```

Macros are text. A `#define` replaces words before the compiler sees them. This block breaks the Norm on purpose (MACRO_FUNC_FORBIDDEN); the lesson is `cc -E`, which prints what the compiler receives:

```
#include <stdio.h>

#define SQUARE(x) x * x

int main(void)
{
    printf("%d\n", SQUARE(1 + 2));
    return (0);
}


$ cc -Wall -Wextra -Werror square.c -o square && ./square
5
$ cc -E square.c | tail -n 3
 printf("%d\n", 1 + 2 * 1 + 2);
 return (0);
}
```

Written `((x) * (x))`, it prints 9. `SQUARE(i++)` would increment `i` twice in one expression, and both compilers refuse it under `-Werror`.

Objects and `nm`. `cc -c f.c` stops at the object file `f.o`; linking objects makes the program. `nm` lists symbols: `T` for a function defined here, `U` for one needed from elsewhere, lowercase for one its file keeps to itself (`static`). `nm -u` keeps the `U` lines, the list to hold against your subject's allowed functions — and it shows what the object calls, not what you typed: from `printf("done\n");`, gcc's object calls `puts`.

Libraries. A static library is an archive of objects:

```
$ ar t "$(gcc -print-file-name=libc.a)" | head -n 3
init-first.o
libc-start.o
sysdep.o
```

`-lNAME` asks the linker for `libNAME.a` (or `.so`), `-L DIR` adds a directory to its search, and a library comes after the objects that need it — the `-lm` rule of Reading compiler errors and warnings.

Makefiles. A rule is `target: prerequisites` plus recipe lines, run when the target is missing or older than a prerequisite. Recipe lines start with a TAB; with spaces, make stops at `missing separator (did you mean TAB instead of 8 spaces?)`. In a pattern rule `%.o: %.c`, `$@` is the target and `$<` the first prerequisite. `.PHONY` marks targets that are not files; `make -n` prints commands without running them.

Worked example: sizes

The `sizes` program of Numbers, parsing and overflow in int, in three files: `size_parse.h` (above); `size_parse.c`, its `scale_of` and `parse_size` under `#include <limits.h>` and `#include "size_parse.h"`; `main.c`, its `main` under `#include <stdio.h>` and `#include "size_parse.h"`. The Makefile:

```
PROG = sizes
SOURCES = main.c size_parse.c
OBJECTS = $(SOURCES:.c=.o)
CFLAGS = -Wall -Wextra -Werror

$(PROG): $(OBJECTS)
    $(CC) $(CFLAGS) -o $@ $(OBJECTS)

%.o: %.c size_parse.h
    $(CC) $(CFLAGS) -o $@ -c $<

clean:
    rm -f $(PROG) $(OBJECTS)

.PHONY: clean
```

```
$ make > /dev/null
$ make
make: 'sizes' is up to date.
$ touch size_parse.h
$ make -n
cc -Wall -Wextra -Werror -o main.o -c main.c
cc -Wall -Wextra -Werror -o size_parse.o -c size_parse.c
cc -Wall -Wextra -Werror -o sizes main.o size_parse.o
$ nm main.o
0000000000000000 T main
                 U parse_size
                 U printf
```

The first rule is the default. Each object depends on its source and the header: a touched header rebuilds both, a touched `main.c` only `main.o`. The Makefile never sets `CC`, so make's built-in default applies (`make -p -f /dev/null 2>/dev/null | grep '^CC ='` prints `CC = cc`; the `2>/dev/null` hides make's complaint that the empty makefile has no targets), and `cc` is clang 12 on this machine; after `make clean`, `make CC=gcc` builds with gcc. `nm size_parse.o` shows `T parse_size` and `t scale_of`.

The bug the header prevents — a caller that types the prototype itself, parameters swapped:

```
#include <stdio.h>

int parse_size(int *bytes, char *s);

int main(void)
{
    int n;

    n = 0;
    if (parse_size(&n, "64K"))
        printf("64K -> %d\n", n);
    else
        printf("64K -> invalid\n");
    return (0);
}
```

```
$ cc -Wall -Wextra -Werror wrong.c size_parse.c -o wrong && ./wrong
64K -> invalid
```

No warning: `parse_size` read the bytes of `n` as its text. With `#include "size_parse.h"` in place of line 3, both compilers refuse the call (`incompatible pointer type`).

Questions to ask

- Which file declares each function, and does the file that defines it include that declaration?
- When this header changes, which objects must be rebuilt — and does the Makefile know?
- Which functions does each object call, and is each on your subject's allowed list?

Self-check

Q1. `main.c` and `size_parse.c` both include `size_parse.h`. Does the guard stop the second one?

Answer: end of the chapter

Q2. Why does `size_parse.c` include its own header?

Answer: end of the chapter

Q3. You edit `size_parse.h` and `make` says `'sizes' is up to date`. What is missing?

Answer: end of the chapter

Pitfalls

Input or situation	Classic bug	Defense
A prototype typed in the caller	Links, then misbehaves	One header, included by both sides
A recipe indented with spaces	`missing separator`	Start recipe lines with a TAB
A changed header	Stale objects	Make the header a prerequisite
A file named `clean`	`make clean` does nothing	`.PHONY: clean`

Practice and check

Practice: a `check` target. Add a phony `check` target: it builds `sizes` if needed, runs it on fixed arguments (valid sizes, `INT_MAX`, the first values past it, malformed text) and compares the output with a hand-written `sizes.expected` (`numfmt --from=iec 64K` prints 65536); when they agree, no difference is printed and `make check` exits 0. Check the check: change one byte of `sizes.expected`; `make check; echo $?` must show the difference and a status other than 0.

Self-check answers

> **Answer to Q1**
>
> No: each `.c` file is compiled on its own, with its own copy; the guard stops a second paste into one file. So a header holds declarations — a function defined there is defined in every object, and the link fails.

> **Answer to Q2**
>
> So the compiler checks the definition against the declaration callers use: a mismatch fails to compile instead of misbehaving.

> **Answer to Q3**
>
> The header is not a prerequisite of the objects, so nothing looks newer than them.

File descriptors and read

C 10 is where your programs open files and read them.

Concepts

The descriptor table. A file descriptor is a small `int` indexing the process's open files; 0, 1 and 2 start open (Output and write). `open` returns -1 or "the lowest-numbered file descriptor not currently open for the process" (`man 2 open`); `close` frees it.

`read` returns a count. `read(fd, buf, n)` returns the bytes copied, 0 at end of file, -1 on error — and "It is not an error if this number is smaller than the number of bytes requested" (`man 2 read`). A pipe hands over what has arrived; a terminal hands over a line at each Enter, and Ctrl-D at the start of a line makes `read` return 0. No terminator is added: the count is the length.

`errno`. A failing call leaves a code in `errno` ; `strerror(errno)` makes it text. It "is significant only when the return value of the call indicated an error" (`man 3 errno`), and the next call may change it.

Worked example: linecount

Spec: `linecount [FILE]` prints the number of `\n` bytes in FILE, or standard input without an argument, reading 4096 bytes at a time (up to `INT_MAX` lines). A failed `open` or `read` prints `linecount: cannot open 'FILE':` (or `cannot read`), the reason and `\n` on standard error, exit 1; extra arguments print `usage: linecount [FILE]` there, exit 2.

```
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#ifndef CHUNK
# define CHUNK 4096
#endif

static int  count_lines(int fd, int *lines)
{
    char    buf[CHUNK];
    ssize_t got;
    ssize_t i;

    *lines = 0;
    got = read(fd, buf, CHUNK);
    while (got > 0)
    {
        i = 0;
        while (i < got)
        {
            if (buf[i] == '\n')
                *lines = *lines + 1;
            i++;
        }
        got = read(fd, buf, CHUNK);
    }
    if (got == -1)
        return (-1);
    return (0);
}

static int  fail(char *what, char *name, int fd)
{
    fprintf(stderr, "linecount: cannot %s '%s': %s\n",
        what, name, strerror(errno));
    if (fd > 0)
        close(fd);
    return (1);
}

int main(int argc, char **argv)
{
    int     fd;
    int     lines;
    char    *name;

    if (argc > 2)
    {
        fprintf(stderr, "usage: linecount [FILE]\n");
        return (2);
    }
    fd = 0;
    name = "standard input";
    if (argc == 2)
    {
        name = argv[1];
        fd = open(name, O_RDONLY);
    }
    if (fd == -1)
```

```
        return (fail("open", name, fd));
    if (count_lines(fd, &lines) == -1)
        return (fail("read", name, fd));
    if (fd > 0)
        close(fd);
    printf("%d\n", lines);
    return (0);
}
```

```
$ cc -Wall -Wextra -Werror -g linecount.c -o linecount
$ printf 'one\ntwo\nthree\n' > three.txt
$ ./linecount three.txt
3
$ printf 'a\nb\n' | ./linecount
2
$ ./linecount nosuch.txt; echo "exit $?"
linecount: cannot open 'nosuch.txt': No such file or directory
exit 1
$ ./linecount /; echo "exit $?"
linecount: cannot read '/': Is a directory
exit 1
```

`count_lines` scans `got` bytes, never `CHUNK`: the last chunk is usually short. `ssize_t` is the signed type `read` returns (`man 2 read`), wide enough for a count and for -1. A directory opens read-only without complaint; `read` then fails with `EISDIR`. `fail` evaluates `strerror(errno)` before `close` can touch `errno`. Descriptor 0 stays open: this program did not open it.

Each `read` is a system call; the catchpoint recipe of Output and write counts them, and `-DCHUNK=1` builds a 1-byte buffer. Save as `reads.gdb`:

```
catch syscall read
commands
silent
continue
end
run three.txt > /dev/null
info breakpoints
```

A call is two hits, entry and return (gdb's start-up warning trimmed):

```
$ cc -Wall -Wextra -Werror -g -DCHUNK=1 linecount.c -o linecount1
$ gdb -q -batch -x reads.gdb ./linecount | grep 'already hit'
    catchpoint already hit 6 times
$ gdb -q -batch -x reads.gdb ./linecount1 | grep 'already hit'
    catchpoint already hit 32 times
```

Three calls against sixteen. Both runs include the final `read` returning 0, and one before `main`: a `bt` at the first catch shows the dynamic loader reading `libc.so.6`.

`valgrind --track-fds=yes ./linecount three.txt` reports `FILE DESCRIPTORS: 3 open (3 std) at exit.` Without `main`'s final `close`: `4 open (3 std)`, then `Open file descriptor 3: three.txt`, opened at `main (linecount.c:60)`.

Questions to ask

- What does your loop do with a `read` that returns fewer bytes than you asked for?
- Which return value ends the loop, which is an error, and what does your subject want printed then?
- Does the program behave the same on a file, a pipe, a terminal and an empty input?

Self-check

Q1. A subject says to read standard input. When do you stop, and what do you trust?

Answer: end of the chapter

Q2. Why does `./linecount /` fail at `read` and not at `open`?

Answer: end of the chapter

Q3. Make a 1000-byte file (`head -c 1000 /dev/zero | tr '\0' x > k.txt`) and put `run k.txt` in `reads.gdb`. Before running it, predict how many `read` calls `linecount` and `linecount1` make. Then run it.

Answer: end of the chapter

Pitfalls

Input or situation	Classic bug	Defense
A short `read`	Processing the whole buffer	Process the count returned
`read` returns -1	Used as a count	Loop while `> 0`; handle -1
A directory as input	Only `open` is checked	Check `read` too
One `open` per argument	Descriptors leak	`close`; `--track-fds=yes`

Practice and check

Practice: longest_line. Write `longest_line [FILE]`: it prints the length in bytes of the longest line of FILE or standard input (bytes before each `\n`; a last line without `\n` counts), reading with `read`. A line can be longer than your buffer. Check it against `awk`:

```
printf '%010000d\n' 0 > long.txt
printf 'ab\nno newline after this one' > last.txt
: > empty.txt
for f in long.txt last.txt empty.txt; do
    awk '{ if (length > m) m = length } END { print m + 0 }' "$f"
done
```

It prints 10000, 25 and 0. A `-DCHUNK=1` build must agree.

Self-check answers

> **Answer to Q1**
>
> Stop when `read` returns 0, or -1 — an error, not a count; what to print then is the subject's call. Trust only the returned count: never a terminator you did not place, never one call for the whole input.

> **Answer to Q2**
>
> Opening a directory read-only succeeds; reading it as bytes fails with `EISDIR`.

> **Answer to Q3**
>
> 3 and 1002 (6 and 2004 hits): the loader's one read, then one per chunk, then the final 0. The count printed is the same; the cost is not.

Part V — Data structures and algorithms

This part passes functions as values, puts data in order, links records into structures, and reads little languages.

Function pointers

C 11 is where functions become arguments; C 12 and C 13 keep passing them.

Concepts

A function has an address. A function's name without parentheses is its address. Read declarations inside-out: in `int (*f)(int)`, `f` is a pointer to a function taking an `int` and returning `int`; `int *f(int)` is a function returning `int *`. Call through it like any function: `f(3)`. A `typedef` names the type, in a header: norminette refuses one in a `.c` file (FORBIDDEN_TYPEDEF).

A callback has a contract. The receiver decides when to call it; a convention decides what its result means. `man 3 qsort` states one: the comparison function returns "an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second." A subject may define another: read it there, test it on two elements.

Compare, don't subtract. `return (a - b);` overflows on distant operands: under `-fsanitize=undefined`, `INT_MAX` and -1 give `runtime error: signed integer overflow: 2147483647 - -1 cannot be represented in type 'int'` and a result of -2147483648. `<` and `>` cannot overflow.

Tables of functions. An array of function pointers maps a small number to a behavior: index it with a command code, call the entry.

Worked example: iterate_until

Spec: `int iterate_until(int x, t_step step, t_test stop, int limit)` replaces `x` with `step(x)` until `stop(x)` holds and returns the number of steps, or -1 once `limit` steps pass. The driver tries two steps: replace a number with the sum of the squares of its digits, until 1 or 4; and with the sum of its digits, until below ten.

```
#ifndef ITERATE_H
# define ITERATE_H

typedef int (*t_step)(int);
typedef int (*t_test)(int);

int iterate_until(int x, t_step step, t_test stop, int limit);

#endif
```

```
#include "iterate.h"

int iterate_until(int x, t_step step, t_test stop, int limit)
{
    int steps;

    steps = 0;
    while (!stop(x))
    {
        if (steps == limit)
            return (-1);
        x = step(x);
        steps++;
    }
    return (steps);
}
```

`main.c`, the test driver:

```
#include <stdio.h>
#include "iterate.h"

int square_digits(int n)
{
    int sum;

    sum = 0;
    while (n > 0)
    {
        sum = sum + (n % 10) * (n % 10);
        n = n / 10;
    }
    return (sum);
}

int one_or_four(int n)
{
    return (n == 1 || n == 4);
}

int digit_sum(int n)
{
    int sum;

    sum = 0;
    while (n > 0)
    {
        sum = sum + n % 10;
        n = n / 10;
    }
    return (sum);
}

int below_ten(int n)
{
    return (n < 10);
}

int main(void)
{
    t_step  step;
    t_test  stop;

    step = square_digits;
    stop = one_or_four;
    printf("squares 7: %d\n", iterate_until(7, step, stop, 1000));
    printf("squares 1: %d\n", iterate_until(1, step, stop, 1000));
    printf("squares 27: %d\n", iterate_until(27, step, stop, 1000));
    printf("squares 27, limit 5: %d\n", iterate_until(27, step, stop, 5));
    step = digit_sum;
    stop = below_ten;
    printf("digits 9875: %d\n", iterate_until(9875, step, stop, 1000));
    return (0);
}
```

```
$ cc -Wall -Wextra -Werror -g iterate.c main.c -o iter
$ ./iter
squares 7: 5
squares 1: 0
squares 27: 10
squares 27, limit 5: -1
digits 9875: 3
```

`step = square_digits;` stores an address; `iterate_until` never knows what it calls. `stop` is tested before each step, so from 1 the answer is 0; `limit` turns a loop that might not end into -1. From 7 the squares go 49, 97, 130, 10, 1; the digit sum goes 9875, 29, 11, 2. A sum of digit squares is at most 810 (ten digits, 81 each), so neither step leaves `int`.

Set `stop = NULL;` on line 46 of `main.c`, rebuild, and run under gdb (start-up lines trimmed):

```
$ gdb -q ./iter
Reading symbols from ./iter...
(gdb) run
Program received signal SIGSEGV, Segmentation fault.
0x0000000000000000 in ?? ()
(gdb) bt
#0  0x0000000000000000 in ?? ()
#1  0x0000000000401156 in iterate_until (x=7, step=0x4011b0 <square_digits>,
    stop=0x0, limit=1000) at iterate.c:8
#2  0x00000000004012ec in main () at main.c:47
```

Frame #0 at address 0, named `??`, is a call through a NULL function pointer; frame #1 shows `stop=0x0`.

Questions to ask

- What does the callback receive, what must it return, and where is that written?
- Who calls the callback, how many times, and in what order?
- Can the pointer be NULL, and can the callback's arithmetic overflow?

Self-check

Q1. Read `int (*cmp)(char *, char *)` aloud.

Answer: end of the chapter

Q2. A comparator returns its first argument minus its second. Ascending or descending — and what goes wrong with `INT_MAX` and -1?

Answer: end of the chapter

Q3. With `step = NULL;` on line 45 of `main.c` instead, frame #1 reads `iterate_until (x=7, step=0x0, stop=0x401210 <one_or_four>, limit=1000) at iterate.c:12` (your addresses will differ). Which call crashed, and why was it not `stop(x)` on line 8?

Answer: end of the chapter

Pitfalls

Input or situation	Classic bug	Defense
A comparator by subtraction	Overflow flips the sign	Compare with `<` and `>`
A convention misread	Output exactly reversed	Test two elements first
`int *f(int)` meant as a pointer	Declares a function	Write `int (*f)(int)`
`f()` passed instead of `f`	Passes a result	Pass the name alone

Practice and check

Practice: the peak. Give `iterate_until` an `int *peak` out-parameter that receives the largest value `x` takes, the start included. That makes five parameters, one over the Norm's limit (TOO_MANY_ARGS): decide what to group or split first. With the digit-square step, from 1 the peak is 1 and from 7 it is 130; with the digit sum, from 9875 it is 9875; and with the digit-square step from 27:

```
awk 'BEGIN { n = 27; m = n; while (n != 1 && n != 4) {
    s = 0; while (n > 0) { d = n % 10; s += d * d; n = int(n / 10) }
    n = s; if (n > m) m = n }; print m }'
```

It prints 145.

Self-check answers

> **Answer to Q1**
>
> `cmp` is a pointer to a function that takes two `char *` and returns an `int`.

> **Answer to Q2**
>
> Ascending: negative means the first is smaller. But `INT_MAX - -1` overflows; under UBSan above it gave -2147483648, so `INT_MAX` sorted before -1.

> **Answer to Q3**
>
> `x = step(x);` on line 12: `step` is the NULL pointer. `stop(x)` ran first and returned, because `stop` holds a real function.

Sorting and searching

Putting things in order comes back in several modules — C 01, C 06, C 11, C 12 — each time on a different type. The Concepts need only arrays and loops (Pointers and arrays). The worked example searches records, which are structs, introduced in Structs, lists and trees: if `books[mid].year` is new to you, read that chapter's Structs paragraph first.

Concepts

Invariants. Trust a sorting loop through what stays true after each pass. Insertion sort keeps the prefix `[0, i)` (indexes 0 up to `i`, with `i` itself excluded) sorted: it takes element `i`, shifts each

larger element of the prefix one place right, and drops it into the gap. Bubble sort swaps out-of-order neighbors; after pass `k`, the `k` largest sit at the end. Selection sort swaps the smallest of the rest to the front of the rest.

Cost. An inversion is a pair in the wrong order. Each shift of insertion sort, and each swap of bubble sort, removes exactly one: no moves on sorted input, n(n - 1) / 2 on reversed input.

Stability. A stable sort keeps equal keys in input order. Insertion sort that shifts only strictly greater elements is stable; selection sort's long swap can jump an element past an equal one. `man 3 qsort`: "If two members compare as equal, their order in the sorted array is undefined." `sort -s` is stable: the reference tool when that order matters. A comparator can break ties itself: compare years, and only when they are equal, titles. `sort -k1,1n -k2` is that order's reference, and on the shelf below it matches `sort -k2 | sort -s -n -k1,1`: sort by the minor key first, then stably by the major one.

Binary search. On sorted data, compare with the middle element and keep the half that can still hold the key: at most 4 steps for 8 elements, 20 for a million. On unsorted data the answer is silently wrong. A half-open `[lo, hi)` starts as `[0, n)` and ends empty; `lo + (hi - lo) / 2` cannot overflow where `(lo + hi) / 2` can. Seeking the first index whose key is at least the target — a lower bound — finds the first of equal keys; with several matches, `bsearch` returns one that is "unspecified" (`man 3 bsearch`).

Worked example: find_year

Spec: a `t_book` holds a title and a year. `int find_year(t_book *books, int n, int year)` returns, on an array sorted by year, the lowest index holding `year`, or -1. The driver's shelf is written in year order already, equal years in the order a stable sort leaves them; `./books` prints it, `./books YEAR...` prints each year's index.

```
#ifndef BOOKS_H
# define BOOKS_H

typedef struct s_book
{
    char    *title;
    int     year;
}   t_book;

int find_year(t_book *books, int n, int year);

#endif
```

```
#include <stdio.h>
#include <stdlib.h>
#include "books.h"

int find_year(t_book *books, int n, int year)
{
    int lo;
    int hi;
    int mid;

    lo = 0;
    hi = n;
    while (lo < hi)
    {
        mid = lo + (hi - lo) / 2;
        if (books[mid].year < year)
            lo = mid + 1;
        else
            hi = mid;
    }
    if (lo < n && books[lo].year == year)
        return (lo);
    return (-1);
}

static void dump_shelf(t_book *books, int n)
{
    int i;

    i = 0;
    while (i < n)
    {
        printf("%d %s\n", books[i].year, books[i].title);
        i++;
    }
}

int main(int argc, char **argv)
{
    static t_book   shelf[8] = {{"Foundation", 1951},
    {"More Than Human", 1953}, {"Fahrenheit 451", 1953},
    {"Solaris", 1961}, {"Stranger in a Strange Land", 1961},
    {"Ubik", 1969}, {"The Left Hand of Darkness", 1969},
    {"Neuromancer", 1984}};
    int             i;

    if (argc == 1)
        dump_shelf(shelf, 8);
    i = 1;
    while (i < argc)
    {
        printf("%s -> %d\n", argv[i], find_year(shelf, 8, atoi(argv[i])));
        i++;
    }
    return (0);
}
```

```
$ cc -Wall -Wextra -Werror -g books.c -o books
$ ./books
1951 Foundation
1953 More Than Human
1953 Fahrenheit 451
1961 Solaris
1961 Stranger in a Strange Land
1969 Ubik
1969 The Left Hand of Darkness
1984 Neuromancer
$ ./books 1953 1951 1984 2000
1953 -> 1
1951 -> 0
1984 -> 7
2000 -> -1
```

In `find_year`, a middle year below the target puts the answer right of `mid`; otherwise `mid` may be the first match, and `hi = mid` keeps it. At the end `lo` is the first index whose year is at least the target, or `n` — 8 for 2000, hence `lo < n` first.

Questions to ask

- What stays true after each pass of your loop, and what does it prove at the end?
- Must equal keys keep their input order, and does your algorithm keep it?
- For an absent key, one smaller than all and one larger than all, where do `lo` and `hi` end?

Self-check

Q1. Selection sort on the keys `2a 2b 1` (two equal 2s). Why is the result not stable?

Answer: end of the chapter

Q2. `lo` is 1500000000 and `hi` is 1800000000, both `int`. What does each midpoint formula compute?

Answer: end of the chapter

Q3. `find_year` looks for 1960 on the shelf. Where is `lo` at the end, and why -1?

Answer: end of the chapter

Pitfalls

Input or situation	Classic bug	Defense
Equal keys	One dropped, or seen as out of order	Keep every element; test duplicates
`>=` in the shift test	Equal keys trade places	Shift only greater; check with `sort -s`
`(lo + hi) / 2`	Overflow on large indexes	`lo + (hi - lo) / 2`
`hi = mid - 1` on `[lo, hi)`	A candidate skipped	One interval convention throughout

Practice and check

Practice: sort_books. Write `void sort_books(t_book *books, int n)`, a stable insertion sort by year, ascending. Its driver holds the shelf in this input order: Neuromancer 1984, Ubik 1969, More Than Human 1953, Solaris 1961, The Left Hand of Darkness 1969, Foundation 1951, Stranger in a Strange Land 1961, Fahrenheit 451 1953. `./books -u` prints that order, as `YEAR TITLE` lines; `./books` sorts it first, and must then print the worked example's listing line for line. The reference is `sort -s` on the first field of the unsorted listing:

```
./books -u | sort -s -n -k1,1 > want.txt
./books > got.txt
diff -u got.txt want.txt && echo SAME
```

Without `-s`, `sort` breaks ties on whole lines: `Fahrenheit 451` moves before `More Than Human`, and `The Left Hand of Darkness` before `Ubik`.

Then count: add a shift counter to `sort_books`, and write a bubble sort over the same array that counts its swaps. On the input order, both counts must equal the number of inversions — pairs where the earlier book has the later year:

```
./books -u | awk '{ y[NR] = $1 } END {
    for (i = 1; i <= NR; i++) for (j = i + 1; j <= NR; j++)
        if (y[i] > y[j]) c++
    print c + 0 }'
```

It prints 19. A sorted shelf gives 0, and the bubble sort's output must pass the `sort -s` recipe.

Self-check answers

> **Answer to Q1**
>
> The 1 swaps with the first element, so `2a` jumps to the end: `1 2b 2a`, past an equal key.

> **Answer to Q2**
>
> `lo + hi` would be 3300000000, past `INT_MAX` (2147483647): signed overflow, which is undefined. `hi - lo` is 300000000, so `lo + (hi - lo) / 2` gives 1650000000 and never leaves `int`.

> **Answer to Q3**
>
> At 3, the first year at least 1960 (Solaris, 1961), which is not 1960.

Structs, lists and trees

C 08 is where structs arrive; C 12 links them into lists and C 13 into trees.

Concepts

Structs. The Norm wants `typedef struct s_name { … } t_name;` in a header, never in a `.c` file (FORBIDDEN_TYPEDEF, as in Function pointers); the listings below keep theirs in the `.c` file only

to fit one block. `e->size` is `(*e).size`. gdb's `ptype /o t_entry` shows a 4-byte hole after `size`, 32 bytes for 28 of fields: allocate with `sizeof`, never your own sum.

Linked lists. Each node points to the next; the last holds NULL, and an empty list is a NULL head. Three laws: a change to where the list starts must reach the caller, through the head's address or a returned new head, as the prototype decides; never read a node after freeing it; when inserting, point the new node at its successor before anything points at it. A circular list's last node points at the first.

Trees. A tree is a node whose children are trees, so recursion fits, with NULL as the base case; first-child and next-sibling pointers hold any number of children. A binary search tree keeps smaller keys left and larger right, so a search follows one path: about log2(n) steps when balanced, n when sorted input made it a list.

Worked example: josephus

Spec: `josephus N K` seats N people, numbered from 1, in a circle and, counting from seat 1, removes every K-th seat in turn, printing each number on its own line; the survivor comes last. N and K are 1 to 1000, else `usage: josephus N K` goes to stderr, exit status 2. The seats live in one block from one `malloc`; if it fails, the exit status is 1.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct s_seat
{
    int             number;
    struct s_seat   *next;
}   t_seat;

t_seat  *build_circle(int n)
{
    t_seat  *seats;
    int     i;

    seats = malloc(sizeof(t_seat) * n);
    if (seats == NULL)
        return (NULL);
    i = 0;
    while (i < n)
    {
        seats[i].number = i + 1;
        seats[i].next = &seats[(i + 1) % n];
        i++;
    }
    return (seats);
}

void    eliminate(t_seat *last, int k)
{
    t_seat  *prev;
    t_seat  *gone;
    int     count;

    prev = last;
    while (prev->next != prev)
    {
        count = 1;
        while (count < k)
        {
            prev = prev->next;
            count++;
        }
        gone = prev->next;
        prev->next = gone->next;
        printf("%d\n", gone->number);
    }
    printf("%d\n", prev->number);
}

int usage(void)
{
    fprintf(stderr, "usage: josephus N K\n");
    return (2);
}

int main(int argc, char **argv)
{
    t_seat  *seats;
    int     n;
    int     k;

    if (argc != 3)
```

```
        return (usage());
    n = atoi(argv[1]);
    k = atoi(argv[2]);
    if (n < 1 || n > 1000 || k < 1 || k > 1000)
        return (usage());
    seats = build_circle(n);
    if (seats == NULL)
        return (1);
    eliminate(&seats[n - 1], k);
    free(seats);
    return (0);
}
```

```
$ gcc -Wall -Wextra -Werror -g josephus.c -o josephus
$ ./josephus 7 3 | cat -e
3$
6$
2$
7$
5$
1$
4$
$ ./josephus 0 3; echo $?
usage: josephus N K
2
```

`build_circle` makes every seat with one `malloc`, so one call can fail and nothing needs freeing when it does. `&seats[(i + 1) % n]` links each seat to the next, the last back to the first, so the seats form a circle by pointer although they sit side by side. `eliminate` starts at the last seat, keeps `prev` one seat behind the count and unlinks the K-th seat before printing it; an unlinked seat stays in the block, and `main` frees the block once, after the last read.

The survivor has a known recurrence, J(1) = 0 and J(i) = (J(i − 1) + K) mod i, survivor J(N) + 1; this loop prints nothing while `awk`'s answer agrees:

```
for n in 1 2 7 41 1000; do for k in 1 2 3 1000; do
  w=$(awk -v n=$n -v k=$k 'BEGIN { for (i = 2; i <= n; i++) j = (j + k) % i; print j + 1 }')
  [ "$(./josephus $n $k | tail -n 1)" = "$w" ] || echo "FAIL n=$n k=$k"
done; done
```

`valgrind --leak-check=full ./josephus 7 3` ends with `All heap blocks were freed -- no leaks are possible`.

Worked example: total_size and print_tree

Spec: a directory tree; each entry has a name, a size (0 for a directory), a first child and a next sibling. `total_size(e)` sums `e` and all below it, 0 for NULL; `print_tree(e, depth)` prints each entry before its children, indented two spaces per level, with its total. Totals fit in `int`.

```c
#include <stdio.h>

typedef struct s_entry
{
    char            *name;
    int             size;
    struct s_entry  *child;
    struct s_entry  *sibling;
}   t_entry;

int total_size(t_entry *e)
{
    int     total;
    t_entry *child;

    if (e == NULL)
        return (0);
    total = e->size;
    child = e->child;
    while (child != NULL)
    {
        total = total + total_size(child);
        child = child->sibling;
    }
    return (total);
}

void    print_tree(t_entry *e, int depth)
{
    t_entry *child;

    if (e == NULL)
        return ;
    printf("%*s%s (%d)\n", depth * 2, "", e->name, total_size(e));
    child = e->child;
    while (child != NULL)
    {
        print_tree(child, depth + 1);
        child = child->sibling;
    }
}

void    set_entry(t_entry *e, char *name, int size)
{
    e->name = name;
    e->size = size;
    e->child = NULL;
    e->sibling = NULL;
}

int main(void)
{
    t_entry e[5];

    set_entry(&e[0], "project", 0);
    set_entry(&e[1], "src", 0);
    set_entry(&e[2], "main.c", 1200);
    set_entry(&e[3], "parse.c", 2300);
    set_entry(&e[4], "big.bin", 4096);
    e[0].child = &e[1];
    e[1].child = &e[2];
    e[2].sibling = &e[3];
```

```
    e[1].sibling = &e[4];
    print_tree(&e[0], 0);
    printf("total of NULL: %d\n", total_size(NULL));
    return (0);
}
```

```
$ ./tree | cat -e
project (7596)$
  src (3500)$
    main.c (1200)$
    parse.c (2300)$
  big.bin (4096)$
total of NULL: 0$
```

Both start with the NULL base case; recursion goes down, the `while` across siblings. `%*s` pads `""` to `depth * 2` spaces (`man 3 printf`). Calling `total_size` on every entry sums each size once per ancestor, a real cost on a deep tree.

Questions to ask

- Who frees each node on every path out, failures included — where your subject allows `free` ?
- Can this operation move where the structure starts, and how does the caller learn of it?
- After this `free` , which pointer do you still need?

Self-check

Q1. What does the arrow in `e->child` do, and when is a dot enough?

Answer: end of the chapter

Q2. Why would a function that may remove the first node take the head pointer's address?

Answer: end of the chapter

Q3. `eliminate` unlinks a seat and moves on. Why not `free(gone)` there, one `free` per seat?

Answer: end of the chapter

Pitfalls

Input or situation	Classic bug	Defense
Empty list or tree (NULL)	Dereferencing it unchecked	NULL is a valid structure: test it first
Removing the first node	The caller's head points at freed memory	Pass the head's address, or return the new head
A circular list	A walk waits for a NULL that never comes	Stop back at the start; break it before freeing

Practice and check

Practice: largest_file. On that tree, print the path of the largest leaf (no child), names joined by `/`, then `\n`; a tie goes to the leaf `print_tree` lists first; a NULL root prints nothing. Expect `project/big.bin`, and `project/src/parse.c` once `big.bin` weighs 2300; check each run with `cat -e`.

Self-check answers

> **Answer to Q1**
>
> It follows the pointer, then takes the field: `(*e).child`. With the struct itself (`e[0].child`), the dot does it.

> **Answer to Q2**
>
> The caller's head pointer must then change, and a copy of it cannot. Returning the new head is the other way; the prototype says which.

> **Answer to Q3**
>
> `free` takes only an address that `malloc` returned, and the seats share one block: `gone` points inside it. A copy that freed `gone` aborted with `double free or corruption (out)` (exit status 134), and valgrind reported `Invalid free()`, `32 bytes inside a block of size 112 alloc'd`. One block, one `free`, once nothing reads the seats.

Parsers and state machines

Rush 01, Rush 02 and BSQ hand you input with rules to check.

Concepts

Grammar and state. A grammar says which tokens exist and which sequences are legal. State is what the next token's meaning depends on — a stack and its count, a position in a field, a flag — kept in named variables with a legal start.

The error contract is the subject's. Where it says bad input prints `Error`, every failed guard leads there; where it promises valid input or says to ignore unknown characters, an `Error` it never asked for is wrong output. A late error leaves two policies: validate first (only the error prints) or stream and stop (earlier output stays).

Why stacks. Nested input closes in reverse order, and a stack returns the most recent value first.

Worked example: stacktoy

Spec: `stacktoy CODE` runs CODE, a string in a four-symbol language: a digit pushes its value, `d` pushes a copy of the top, `o` (over) pushes a copy of the second value from the top, and `p` pops the top and prints it on its own line on standard output. An unknown symbol, too few values or a full stack (4096 values) stops the run: it writes `stacktoy: symbol N: REASON` (N counted from 1; REASON is `unknown symbol`, `too few values` or `stack full`) and a newline to standard error, and exits 1; output already printed stays. Any number of arguments other than one writes `usage: stacktoy CODE` to standard error and exits 2.

```
#include <stdio.h>
#include <unistd.h>

#define STACK_MAX 4096

char    *op_push(int *stack, int *top, int value)
{
    if (*top == STACK_MAX)
        return ("stack full");
    stack[*top] = value;
    *top = *top + 1;
    return (NULL);
}

char    *op_print(int *stack, int *top)
{
    char    digit;

    if (*top == 0)
        return ("too few values");
    *top = *top - 1;
    digit = (char)('0' + stack[*top]);
    write(1, &digit, 1);
    write(1, "\n", 1);
    return (NULL);
}

char    *step(int *stack, int *top, char symbol)
{
    if (symbol >= '0' && symbol <= '9')
        return (op_push(stack, top, symbol - '0'));
    if (symbol == 'p')
        return (op_print(stack, top));
    if (symbol == 'd' && *top >= 1)
        return (op_push(stack, top, stack[*top - 1]));
    if (symbol == 'o' && *top >= 2)
        return (op_push(stack, top, stack[*top - 2]));
    if (symbol == 'd' || symbol == 'o')
        return ("too few values");
    return ("unknown symbol");
}

char    *run(char *code, int *pos)
{
    int     stack[STACK_MAX];
    int     top;
    char    *reason;

    top = 0;
    reason = NULL;
    *pos = 0;
    while (code[*pos] != '\0' && reason == NULL)
    {
        reason = step(stack, &top, code[*pos]);
        *pos = *pos + 1;
    }
    return (reason);
}

int main(int argc, char **argv)
{
    char    *reason;
```

```
    int     pos;

    if (argc != 2)
    {
        fprintf(stderr, "usage: stacktoy CODE\n");
        return (2);
    }
    reason = run(argv[1], &pos);
    if (reason == NULL)
        return (0);
    fprintf(stderr, "stacktoy: symbol %d: %s\n", pos, reason);
    return (1);
}
```

Both compilers build it clean with `-Wall -Wextra -Werror`; norminette reports only the missing 42 header.

```
$ ./stacktoy 12op
1
$ ./stacktoy 5px; echo $?
5
stacktoy: symbol 3: unknown symbol
1
$ ./stacktoy ""; echo $?
0
$ ./stacktoy; echo $?
usage: stacktoy CODE
2
```

`top` counts the live values and names the next free slot. Each helper guards before it touches the array: `step` checks the count before `d` or `o` reads below the top, and `op_push` refuses a full stack. The helpers return a reason or NULL; `run` counts the symbols, so it knows N, and `main` alone writes the message. The 25-line limit pushed the helpers out of `run`. The empty program breaks no rule; `5px` shows the policy, stream and stop.

Questions to ask

- What is the state, its legal start, and its legal end?
- Before each token acts, what must be true of the state?
- Does the subject say bad input exists, and what it produces?

Self-check

Q1. Your code pops, then checks for an empty stack. Which inputs break it, and which tool sees it?

Answer: end of the chapter

Q2. What does `./stacktoy 12op9x` print, and what would a validate-first version print?

Answer: end of the chapter

Q3. What would change if `op_print` wrote its own message to standard error instead of returning a reason?

Answer: end of the chapter

Pitfalls

Input or situation	Classic bug	Defense
More pops than pushes	The index drops below 0	Guard before every pop
Input longer than the stack	A push writes past the array	Guard every push against the capacity
Empty input	Treated as an error, or prints garbage	Is the empty sequence legal? Test it

Practice and check

Check the worked example with hostile inputs, then its capacity (`printf '%04096d' 7` writes 4096 digits ending in `7`). Standard output carries only the `p` lines, so the loop keeps the status, standard output and standard error apart:

```
for a in "" "p" "o" "5dp" "12opp" "9x" "5 p"; do
    ./stacktoy "$a" > out.txt 2> err.txt
    s=$?
    o=$(paste -s -d ' ' out.txt)
    printf '[%s] status %s, out [%s], err [%s]\n' "$a" "$s" "$o" "$(cat err.txt)"
done
./stacktoy "$(printf '%04096d' 7)p"
./stacktoy "$(printf '%04097d' 7)p"; echo $?
```

```
[] status 0, out [], err []
[p] status 1, out [], err [stacktoy: symbol 1: too few values]
[o] status 1, out [], err [stacktoy: symbol 1: too few values]
[5dp] status 0, out [5], err []
[12opp] status 0, out [1 2], err []
[9x] status 1, out [], err [stacktoy: symbol 2: unknown symbol]
[5 p] status 1, out [], err [stacktoy: symbol 2: unknown symbol]
7
stacktoy: symbol 4097: stack full
1
```

Practice: is_ipv4. Write `int is_ipv4(char *s)` with an explicit state variable: 1 for four decimal parts 0 to 255 joined by single dots — no empty part, no leading zero in a multi-digit part, no sign, no space, nothing else — and 0 otherwise. A driver `./ipv4 STRING` prints the result and a newline; `""` is a case, so the table splits on `|` :

```
cat > cases.txt <<'EOF'
1.2.3.4|1
255.255.255.255|1
0.0.0.0|1
256.1.1.1|0
1.2.3|0
1..2.3|0
01.2.3.4|0
1.2.3.4.|0
|0
EOF
while IFS='|' read -r s want; do
  got=$(./ipv4 "$s")
  [ "$got" = "$want" ] || echo "FAIL [$s] got $got want $want"
done < cases.txt
```

Silence is a pass; a driver that always prints `1` gets six `FAIL` lines.

Self-check answers

> **Answer to Q1**
>
> Any input needing more values than were pushed, such as `p` first: a read below index 0, undefined behavior. A broken copy of `stacktoy` still printed `stacktoy: symbol 1: too few values` in a plain run, and `valgrind` was silent (it does not watch stack arrays); ASan stopped at the read (gcc: `stack-buffer-overflow`, clang 12: `stack-buffer-underflow`).

> **Answer to Q2**
>
> `1` on standard output, then `stacktoy: symbol 6: unknown symbol` on standard error, exit status 1: `p` printed before the scan reached `x`. Validate-first would print the error line alone.

> **Answer to Q3**
>
> It cannot know N, which only `run` counts, and `main` would still write its own line: two messages for one failure, in two formats. One place that writes keeps one line per failure.

Part VI — Appendices

These are cards to keep beside the keyboard; the chapters hold the why.

Appendix A — Pre-submit checklist

- Names copied from the subject, not retyped?
- Prototypes match the subject character for character?
- No test `main` in a file of functions?
- Every library call allowed? `nm -u file.o` lists them, plus the compiler's own (`__stack_chk_fail`).
- `gcc` and `clang` , `-Wall -Wextra -Werror` (`-c` for functions): silent?
- `norminette` prints `OK!` for every file?
- Sanitizer build and `valgrind` clean on your tests?
- Every subject example matches byte for byte under `cat -e` ?
- No arguments, `""` , empty input: no crash?
- `git status` lists exactly the subject's files?

Appendix B — Universal edge-case matrix

Rows for any C task.

Input or situation	Classic bug	Defense
`""`	Reading meaning into `str[0]`	Zero length falls through your loops
`" "`	A word scanner runs past the terminator	Every scanning loop tests for `'\0'`
`NULL` , above all the empty list	Dereferencing it unchecked	An empty list is a list; elsewhere, the subject says if `NULL` can arrive
`0`	`while (n > 0)` never runs, so nothing prints	Run 0 itself and read what prints
`-2147483648` (`INT_MIN`)	Negating it overflows: `INT_MAX` is 2147483647	Run that one value through your code, under UBSan too
A multi-sign string like `"--+42"`	Assuming libc's rules where the subject sets its own	Sign rules are the subject's
`malloc` returns `NULL`	Using the result unchecked	Return what the subject specifies; free what you built where it allows `free`
The end of an array of strings	No terminating `NULL`	A slot for the sentinel when the subject expects one
`./prog` alone (`argc == 1`)	Using `argv[1]` as a string while it is `NULL`	Check `argc` first; the subject sets the output
`./prog ""`	`argc` is 2, but the argument is empty	Row one, applied to `argv[1]`

Input or situation	Classic bug	Defense
No trailing newline	Right on screen, no final `$` under `cat -e`	A subject's example is exact, newline included
Off by one at the terminator	`'\0'` never written, or written past the end	Allocate length + 1; re-read every bound

Appendix C — GDB and LLDB card

Every line ran on the specimen of Debuggers: gdb and lldb as peers, under gdb 12.1 and lldb 12.0.1 (installed as `lldb-12`).

Task	gdb	lldb
Run with arguments	`run banana a`	`run banana a`
Break at a function, a line	`break strip_letter`, `break strip_letter.c:19`	`b strip_letter`, `b strip_letter.c:19`
Break if a condition holds	`break strip_letter.c:21 if i == 3`	`br s -f strip_letter.c -l 21 -c 'i == 3'`
Step over, into; resume	`next`, `step`, `continue`	`next`, `step`, `continue`
Print, in hex	`print j`, `print/x len`	`p j`, `p/x len`
Raw bytes	`x/16xb out`	`x -s1 -fx -c16 out`
All locals	`info locals`	`frame variable`
Call stack, move up	`bt`, `up`, `frame 1`	`bt`, `up`, `frame select 1`
Stop when a variable changes	`watch j`	`watchpoint set variable j`
Stop at a system call	`catch syscall write`	—
Show source	`list`	`source list`

To stop on every pass of a loop, break on a line of its body: on the specimen as printed, `break 21` stopped four times on `world o`, once per pass, under both compilers. A breakpoint on the `while` line itself (`break 19`) stopped once in a gcc build and five times in a clang one.

Segfault triage in one line (the full drill closes Debuggers: gdb and lldb as peers): reproduce with the exact arguments, rebuild with `-g`, `run` them in the debugger, read your innermost frame in `bt` (`0x0` is often the story), then `print` that line's variables.

Appendix D — Compiler and sanitizer card

Flag	What it does
`-Wall -Wextra`	common warnings; `-Wextra` adds sign-compare and unused-parameter
`-Werror`	any warning stops the build
`-g`	file and line numbers for the debuggers, valgrind and gcc's sanitizer reports; clang 12's sanitizer reports also need a symbolizer (see below)
`-c`, `-o NAME`	compile to a `.o` only; name the output

Flag	What it does
`-E`	preprocess only: see what a macro became
`-fsanitize=address,undefined`	ASan and UBSan; add `-g`
`-fno-sanitize-recover=all`	UBSan exits 1 at its first report
`-lNAME`	link `libNAME` ; gcc wants it after the files that use it

Read `file.c:LINE:COL: error: message [flag]` with its `note:` lines, and fix the first message first: later ones are often its echo.

The message contains	It usually means
`implicit declaration of`	a missing `#include`
`comparison of integer`	`int` against `unsigned` : a sign can flip the test
`non-void function`	a path ends without `return`
`undefined reference to`	linker: declared, never defined — a file or `-l` missing
`multiple definition of`	linker: two definitions, often two `main` s

An ASan report says what (`ERROR: AddressSanitizer: heap-buffer-overflow` , `WRITE of size 1`), where (your first frame) and where the block was born (`allocated by thread T0 here:`). UBSan prints `FILE:LINE:COL: runtime error: …` and carries on. The exit status is whatever the program does next: the overflow toy exited 0, and the division toy then hit SIGFPE (status 136 with gcc; clang's runtime caught the signal and exited 1). With `-fno-sanitize-recover=all` , the first report exits 1. Frames without names mean no symbolizer, as with clang 12 on this guide's machine.

Appendix E — Valgrind decoder

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./prog args
```

Line in the report	What it means	First place to look
`Invalid write of size 1`	a write outside a block you own	the size: a missing `+ 1`
`Invalid read of size 1`	a read outside a block you own	loop bounds; scanning past `'\0'`
`N bytes in 1 blocks are definitely lost`	never freed, unreachable	a `free` on every path out, where `free` is allowed
`Conditional jump or move depends on uninitialised value(s)`	a branch on a variable never assigned	`--track-origins=yes` says where it was born

A clean run of `josephus` , trimmed: one block of seven 16-byte seats plus `printf` 's buffer.

```
$ valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./josephus 7 3 > /dev/null
==23641== HEAP SUMMARY:
==23641==     in use at exit: 0 bytes in 0 blocks
==23641==   total heap usage: 2 allocs, 2 frees, 4,208 bytes allocated
==23641== All heap blocks were freed -- no leaks are possible
==23641== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Appendix F — Reference-tool card

Programs to compare your output with (Testing yourself); each example ran on small files.

Tool	What it shows	Example
`cat -e`	`$` at each line end; `cat -A` adds tabs as `^I`	`cat -e out.txt`
`diff -u`, `cmp`	differing lines (`-` first file, `+` second); the first differing byte	`diff -u got want`
`od -c`, `xxd`, `hexdump -C`	every byte, invisible ones included	`od -c out.txt`
`tr`, `rev`	bytes mapped or deleted; each line reversed	`tr -d a < in.txt`
`sort -s -n -k1,1`	a stable numeric sort on field one	`sort -s -n -k1,1 in.txt`
`wc -l`, `awk`	the count of `\n` bytes; each line's length	`awk '{ print length }' in.txt`
`expr`, `bc`	`/` truncating as in C: `-3`; no size limit	`expr -7 / 2`, `bc -q`
`printf`	hex and octal: `c8 310`	`printf '%x %o\n' 200 200`
`factor`, `numfmt`	`91: 7 13`; `65536`	`factor 91`, `numfmt --from=iec 64K`
`seq`	numbers for a test loop: `0 5 10 15 20`	`seq -s ' ' 0 5 20`

Appendix G — Glossary

- **`-Werror`** — every warning becomes a compile failure.
- **archive** — a `.a` file of object files that the linker searches; `ar t` lists it.
- **base case** — what a recursive function answers without recursing.
- **binary search** — halving a sorted range: about log2(n) steps.
- **`cat -e`** — shows line ends as `$`.
- **comparator** — orders two elements: negative, zero or positive.
- **diff (unified)** — `diff -u`: `-` first file, `+` second.
- **errno** — the error code a failed call leaves; `strerror` names it.
- **exercise** — one task of a module: functions or a whole program.
- **file descriptor** — an `int` naming an open file.
- **gdb**, **lldb** — debuggers: run a program, stop it, inspect it; Appendix C pairs their commands.
- **heap** — `malloc`'s memory, alive until freed.
- **include guard** — `#ifndef` around a header, so it counts once.
- **leak** — heap memory never freed.
- **libc** — the standard C library.
- **linker** — joins object files and libraries, resolving names.
- **Makefile target** — what a rule builds: a file, or a `.PHONY` name.
- **NUL terminator** — the `'\0'` ending a C string.
- **object file** — the `.o` that `-c` writes.
- **out-parameter** — a pointer a function stores a result through.

- **ownership** — who must free a block.
- **parser** — code that checks input against rules.
- **reference tool** — an existing program whose output you trust and compare yours with; Appendix F.
- **sanitizer** — checks compiled into a program, on the inputs you run.
- **segfault** — a crash on memory you do not own.
- **sentinel** — the `NULL` ending an array of pointers.
- **short-circuit** — `&&` and `||` stop once the result is known.
- **stable sort** — keeps equal keys in input order.
- **stack frame** — one call's arguments and locals.
- **state machine** — code whose next move depends on a state it keeps.
- **subject** — the assignment's text, the authority on names and allowed functions.
- **undefined behavior** — an operation C defines no result for, such as signed overflow.
- **vacuous truth** — a claim about every element of an empty set is true: no element can refute it.
- **valgrind** — runs an unchanged program and reports memory errors and leaks; Appendix E.

Appendix H — Sources and trust methodology

Every output here came from the machine named in How to use this guide: each C block meant to build was built with gcc and clang under `-Wall -Wextra -Werror` and run; the deliberately broken specimens were compiled to show exactly the messages printed; each recipe was run as printed under `sh` (dash). Where yours differs, trust your machine and its man pages. Sources:

1. The man pages, sections 1, 2, 3 and 7.
2. The C standard's public C11 draft N1570: 6.3.1 (conversions), 6.5.5 (division truncates toward zero), Annex J.2 (undefined behavior).
3. The GCC manual, "Warning Options"; Clang's "Diagnostic flags in Clang".
4. The GDB manual; LLDB's "GDB to LLDB command map".
5. The Valgrind manual ("Memcheck"); Clang's AddressSanitizer and UndefinedBehaviorSanitizer pages; GCC's `-fsanitize` options.
6. The GNU Make manual.
7. 42's Norm, and norminette's error codes.